

BE Project

CE Department

Project ID: BUKC-CE-2025-07

April 2026



Bahria University
Discovering Knowledge

Emergency Rescue Node Communication System

Muhammad Tayyab

Muhammad Talha

Samra Ali

Department of Computer Engineering

Bahria University, Karachi Campus

Final Year Project Report

Emergency Rescue Node Communication System

By

Muhammad Tayyab

81993

Muhammad Talha

81991

Samra Ali

82008

Project Supervisor: Engr. Usra Sami

Deliverable

Report 1 Volume

Program 1 CD

Bahria University (Karachi Campus)

Submission Performa

Name (1) Muhammad Tayyab

(2) Muhammad Talha

(3) Samra Ali

Address (1) BUKC

(2) BUKC

(3) BUKC

Title of Report: Emergency Rescue Node Communication System

Project Supervisor's Name: Engr. Usra Sami

This report is submitted as per the requirements of the Project in accordance with the rules laid down by Bahria University as part of the requirements for the award of the degree of Bachelor of Engineering. I/we declare that the work presented in this report is my/our own except where appropriate reference or credit is given to the work of others.

Signatures of Students

Date

(1)

.....

(2)

.....

(3)

.....

Signature of Supervisor

Date

.....

.....



Bahria University
Discovering Knowledge

Intellectual Property Right Declaration

We hereby declare that the work carried out under the supervision of **Engr. Usra Sami** titled "**Emergency Rescue Node Communication System**" undertaken in partial fulfillment of Bachelor of Engineering in Computer Engineering is the exclusive property of Bahria University and protected under intellectual property rights laws and conventions. Bahria University asserts legal and beneficial ownership rights in all intellectual property developed because of support either directly from or through Bahria University or created at the request or direction of Bahria University, or developed because of the use of Bahria University resources, including copyright in any material. It can only be considered/used for purposes like extension for further improvement, product development, adoption for commercial/organizational use etc. only with the permission of the University and in accordance with its policies.

The above statements apply to all students and faculty members.

Date: May, 16th 2026

Author(s):

Name: Muhammad Tayyab

Signature: _____

Name: Muhammad Talha

Signature: _____

Name: Samra Ali

Signature: _____

Supervisor(s):

Name: Engr. Usra Sami

Signature: _____



Bahria University
Discovering Knowledge

Anti-Plagiarism Declaration

We hereby declare that the above publication created under the supervision of Engr. Usar Sami having title "Emergency Rescue Node Communication System" is the sole contribution of the author(s) and no part has been reproduced here illegally (cut and paste) which can be considered as plagiarism. All referenced sections were used to argue the point and were properly cited. We will be responsible and liable for any consequences if a violation of this statement is proven.

Date: April, 12th 2026

Author(s):

Name: Muhammad Tayyab

Signature: _____

Name: Muhammad Talha

Signature: _____

Name: Samra Ali

Signature: _____

Acknowledgments

Final Year Project is a demonstration for undergraduate students, which is a combination of teamwork and implementation of theoretical and practical knowledge. It enhances the abilities of students to step up in their field. With this willingness, we affiliated with this project.

In the successful accomplishment of our project, we would like to express our sincere gratitude and appreciation to those people who were actively involved in our project.

Foremost, all thanks to **Allah (S.W.T)** for enabling us to complete this great feat and for helping to make our project successful.

We are highly obliged to sincerely thank our project coordinator **Dr. Rizwan Iqbal** for helping us in managing and other project tasks. We also extend our deepest thanks to the Head of the CE Department, **Dr. Faisal Siddiqui**, for his support and kind cooperation during our difficult phases. Lastly, all our team members express great appreciation and special thanks to our project supervisor Engr. Usra Sami for his guidance, monitoring, and continuous support throughout the project lifecycle with his vast experience and knowledge.

Abstract

Natural disasters keep showing us that there is a problem with how we respond to emergencies. The systems we rely on often fail when we need them the most. This happened in Pakistan during the 2025 floods. Over 4,300 phone towers stopped working in a few days. This meant that millions of people could not call for help [7].

This report is about a system called the Emergency Rescue Node Communication System. It is a kind of network that lets people communicate with each other even when the main systems are down. This system is built using LoRa ESP32 V2 development boards.

The system has a network of devices called rescue nodes. Each node is a computer that can send and receive messages. These nodes are portable so they can be taken to where they're needed. Each node creates its own Wi-Fi network and people can connect to it using their smartphones. When someone connects their browser opens a page where they can send a message for help. The node can even find the persons location using the GPS on their smartphone.

The nodes can talk to each other using a kind of radio message. This means that messages can be sent from one node to another even if they are not close to each other. The messages have all the information, like who is sending the message, where they are and what they need help with.

The system also has a device called a gateway node. This node gets all the messages. Shows them on a screen. It also sends the messages to a website where people can see where the messages are coming from and what help is needed. There is also an app for smartphones that can send messages for help and show peoples locations.

This system is important because it can be used in places where the main systems are down. It is also affordable. Can be set up quickly. This means that it can help people in disaster-affected areas. The system is also good, for the environment. Can help reduce the risk of disasters. It aligns with the United Nations Sustainable Development Goals 3, 11 and 13.

The Emergency Rescue Node Communication System uses LoRa, ESP32. Lora ESP32 V2. It is a mesh network that can send messages in hops. It is an emergency communication system that can be used in disasters. It has a portal and a Wi-Fi hotspot and it can work offline. It can send SOS alerts. Use phone GPS. The system was inspired by the Pakistan Floods. Supports SDG 3 SDG 11 and SDG 13.

Table of Contents

Chapter 1: Introduction.....	1
1.1 Purpose of this Project	1
1.2 Complex Engineering Problem Statement	1
1.3 Goal of this Project	2
1.4 Scope of the Project	3
1.5 SDG Statements	4
1.6 Document Overview	4
1.7 Project Modules	5
1.8 Overview and Design Recommendations	5
1.8.1 Rescue Nodes (Node 1 Node 2):	6
1.8.2 Gateway Node (Node 3):	6
1.8.3 Multi-Hop Communication:	6
1.8.4 Design Recommendations.....	6
Chapter 2: Background and Literature Review.....	8
2.1 Existing Systems.....	8
2.1.1 Existing Systems Description	8
2.1.2 Problems in the Existing Systems.....	9
2.2 Related Work	9
Chapter 3: System Analysis	11
3.1 Work Analysis	11
3.1.1 Work Flow Diagram	11
3.1.2 Work Breakdown Structure	13
3.2 Data Analysis	13
3.2.1 Data Flow Diagram.....	14
3.3 Requirements of the System	15
3.3.1 Clients, Customer and Users.....	15
3.3.2 Resource Requirements.....	15
3.3.3 Data Requirements.....	16
3.3.4 Non-Functional Requirements	17
3.4 Proposed Solution	19
Chapter 4: System Design.....	20
4.1 Project Modules	20
4.2 Design Constraints	20
4.3 Hardware and Software Environment.....	21
4.3.1 Hardware.....	21
4.3.2 Software	23

4.4 Architecture Planning Techniques	24
4.4.1 Algorithm to be used.....	24
4.4.2 Development Method.....	25
4.5 Project Management Strategies.....	25
4.5.1 Task Calendar	26
4.5.2 Project Timeline.....	27
4.5.3 Management of Quality	28
4.5.4 Human Resource Management	28
4.5.5 Risk Management	29
4.6 Data Conversions.....	29
4.7 Application Program Interface.....	30
4.8 User Interface.....	31
4.8.1 Mobile Application	31
4.8.2 Admin Web Portal	33
4.9 Database Design.....	33
4.10 Performance	35
4.11 Architecture Design	35
4.11.1 Logical View.....	36
4.11.2 Hardware Architecture.....	36
4.11.4 Security Architecture	38
4.12 Use-Case Model.....	38
Chapter 5: Implementation	40
5.1 Code Implementation.....	40
5.1.1 Wi-Fi Access Point and Captive Portal.....	40
5.1.2 Phone GPS Capture via Browser Geolocation API	40
5.1.3 SOSPacket Construction and LoRa Transmission.....	40
5.1.4 Multi-Hop Relay Algorithm.....	40
5.1.5 Gateway Forwarding to Dashboard	41
5.2 Hardware Prototype Assembly	41
5.3 LoRa ESP32 V2 Node Configuration.....	42
5.4 Captive Portal SOS Interface	43
5.5 LoRa Mesh Communication Testing.....	45
5.6 Phone GPS Integration Testing.....	45
5.7 Power Supply and Battery Management.....	45
5.8 Database and Dashboard Implementation.....	46
5.9 Android Application Implementation	46
Chapter 6: Testing.....	47
6.1 Purpose of the Test Plan	47
6.2 System Function Testing.....	47

6.2.1 Testing Challenges.....	47
6.2.2 Items for Testing.....	48
6.2.5 Success Verification Criteria	49
6.2.6 Testing Start and Completion Conditions	50
6.2.7 Test Outputs.....	50
6.3 Performance Testing	51
6.3.1 Load Testing	51
6.3.2 Test Risks / Issues	52
6.3.3 Items for Testing.....	52
6.3.4 Test Approaches.....	53
6.3.5 Connectivity and Compliance Conditions	53
6.3.6 Success and Failure Conditions	53
6.4 System Evaluation	54
6.4.1 Potential Testing Challenges.....	54
6.4.2 Items to be Tested	54
6.4.3 Security check.....	55
6.4.4 Functionality check.....	55
6.4.5 Test Regulatory / Mandate Criteria.....	55
6.4.6 Test Pass / Fail Criteria	56
Chapter 7: Impact on Society and Environment	57
7.1 Impact on Society	57
7.2 Impact on Environment.....	57
Chapter 8: Results and Discussion.....	60
8.1 LoRa Mesh Communication Performance.....	60
8.2 Captive Portal SOS Interface Performance.....	60
8.3 Gateway and Dashboard Performance.....	61
8.4 Power and Battery Performance	61
8.5 System-Level Evaluation Summary.....	61
8.6 Discussion	64
Chapter 9: Conclusions and Future Work.....	65
9.1 Conclusion	65
9.2 Future Work.....	66
Chapter 10: References.....	67
Appendices.....	68
Appendix A.....	68
A.1 Frontend Code.....	68
A.1.1 App.tsx — root application component	68
A.1.2 Admin Dash board	70
A.1.3 types/api.ts — shared TypeScript interfaces	72

A.1.4 services/backendService.ts — HTTP client	72
A.2 Backend Code	74
A.2.1 main.py	74
A.2.2 models.py	74
A.2.3 schemas.py	75
A.2.4 routers/users.py	76
A.2.5 routers/admin.py	77
A.3 Embedded Systems Code – Arduino, C++	79
A.3.1 EmergencyNodes	79
A.3.2 GatewayNode	80
A.3.3 bridge.py — gateway USB serial bridge	81
A.4 Android Application Code	83
A.4.1 MainActivity.java — main activity	83
A.4.2 activity_main.xml — main activity layout	85

Table of Figures

Figure No.	Title	Page No.
Fig. 1.1	System Overview — SOS Alert Flow	3
Fig. 1.2	System Overview Data Flow Diagram	7
Fig. 3.1	Over all System Workflow Diagram	12
Fig. 3.2	Data Flow Diagram	14
Fig. 4.1	TP4056 charging circuit image	22
Fig. 4.2	Board Mechanical Dimensions (mm)	23
Fig. 4.3	Project Gantt Chart	27
Fig. 4.4	Mobile Application UI Screen	32
Fig. 4.5	Admin Web Portal Dashboard	33
Fig. 4.6	Database Schema (ER Diagram)	34
Fig. 4.7	System Architecture Diagram	35
Fig. 4.8	Software Architecture Diagram	37
Fig. 4.9	Use Case Diagram (UML)	38
Fig. 5.1	Physical hardware Prototype	42
Fig. 5.2	Wi-Fi LoRa 32 (V2) Pinout Diagram	43
Fig. 5.3	Captive Portal SOS Page on Victim Smartphone	44
Fig. 8.1	Wireless Communication Performance (a, b, c)	61
Fig. 8.2	System Response and Reliability (d, e, g, h)	62
Fig. 8.3	Hardware and Physical Accuracy (f, I)	62

Table of Tables

Table No.	Title	Page No.
Table 1-1	Overview if System Modules	5
Table 3-1	Work Breakdown Structure	13
Table 3-2	Non-Functional Requirements	17
Table 3-3	Functional Requirements	18
Table 4-1	Hardware Components List	22
Table 4-2	Monthly Development Phases and Deliverables	26
Table 4-3	Project Development Phases	27
Table 4-4	Mobile Application Feature Summary	31
Table 4-5	Rescue Dashboard Feature Summary	33
Table 4-6	Database Schema Overview	34
Table 5-1	LoRa Radio Configuration Parameters	42
Table 6-1	Items for Testing	48
Table 6-2	Functional Test Cases and Results	51
Table 7-1	CLO Coverage Mapping Across the Thesis Report	58
Table 8-1	System Performance Evaluation Summary	60

List Of Abbreviations

AP	— Access Point
API	— Application Program Interface
BLE	— Bluetooth Low Energy
BW	—Bandwidth
CR	—Coding Rate
CSS	—Chirp Spread Spectrum
DFD	—Data Flow Diagram
DIO	—Digital Input Output
ESP32	—Espressif Systems Programmable 32-bit Microcontroller
GB	—Gateway Board
GPIO	—General Purpose Input Output
GPS	—Global Positioning System
HTML	—Hypertext Markup Language
I2C	—Inter-Integrated Circuit
ID	—Identifier
IoT	—Internet of Things
IP	—Internet Protocol
ISM	—Industrial Scientific and Medical
JS	—JavaScript
JSON	—JavaScript Object Notation
LiPo	—Lithium Polymer
LoRa	—Long Range
LoRaWAN	—Long Range Wide Area Network
LPWAN	—Low Power Wide Area Network
MCU	—Microcontroller Unit
MHz	—Megahertz
NFR	—Non-Functional Requirement
OLED	—Organic Light Emitting Diode
PCB	—Printed Circuit Board
PostgreSQL	—Post Structured Query Language
RSSI	—Received Signal Strength Indicator
SDK	—Software Development Kit
SOS	—Save Our Souls
SPI	—Serial Peripheral Interface
SSID	—Service Set Identifier
SX1276	—SimTech LoRa Transceiver Model 1276
TX	—Transmit
RX	—Receive
UX	—User Experience
V2	—Version 2
Wi-Fi	—Wireless Fidelity

CHAPTER 1: INTRODUCTION

1.1 Purpose of this Project

This project is about creating an Emergency Rescue Node Communication System. The Emergency Rescue Node Communication System is a system that lets people communicate during emergencies even when they do not have internet. It has an important part: LoRa ESP32 V2 rescue nodes, a special network that lets these nodes talk to each other a simple interface on smartphones that people can use to send SOS messages an Android app and a website that rescue teams can use to monitor everything. A team of Computer Engineering students, from Bahria University, Karachi Campus made this Emergency Rescue Node Communication System under the guidance of Engr. Usra Sami. The main goal of the Emergency Rescue Node Communication System is to help people communicate during disasters when regular phone and internet services are not working. The Emergency Rescue Node Communication System is important because it helps fix the problem of people not being able to communicate during disasters [7][11].

The project uses rescue nodes that make their own Wi-Fi hotspots and send emergency messages over long distances using LoRa radio. This way the project does not need a lot of infrastructure to work which is a problem with other emergency communication systems. The project gets the location of the person in trouble from their smartphone using a tool in the browser. This means we do not need to put a GPS device on each rescue node so we can find the person in trouble very easily and it does not cost a lot.

When someone sends an SOS message it goes from node to node until it reaches a node that rescue teams can use to get the message. They can see the message on a screen or on a website that shows all the messages. This report tells us how the project went from an idea to a working prototype. It shows that we can make a system for emergency communication that works well is easy to make bigger and does not cost much which is very important for areas that have been affected by disasters. The emergency communication system is very important for people, in these areas. The rescue nodes are a big part of the emergency communication system.

1.2 Complex Engineering Problem Statement

Pakistan is a country that has been hit by disasters like floods, earthquakes and cyclones many times. This has shown that the country has a problem with how it responds to emergencies. For example, during the 2025 floods than 4,300 telecom towers were damaged in just a few days. This cut off millions of people from all communication services at a time when they really needed them [7].

The systems that cities use to respond to emergencies are not working well because they need cellular networks, internet and other infrastructure to work. These things often get damaged during disasters.

Emergency communication systems that use cell phones need base stations. Switching centres to work. Internet-based emergency apps need data connectivity. Satellite communication systems do not need infrastructure. They are too expensive and not suitable for many people in developing areas.

All these solutions have a problem. They need infrastructure that often gets destroyed during disasters. So, the main problem is that we do not have a communication system that's affordable, easy to set up and does not need infrastructure. This system should be able to work without networks, internet or power grid, which are often not available during disasters.

The Pakistan emergency communication system should be able to work under the conditions:

- The system must work without depending on networks, internet connectivity or power grid.
- The system must be easy to use by people who're not technicians even in very stressful situations and should not require a lot of steps or special knowledge.
- The system should be able to find the location of victims and send this information without needing GPS hardware, which would make it cheaper and simpler.
- If one part of the network is damaged the system should still be able to send messages through parts.
- The system should be able to work for a time on battery power even in areas without electricity.
- The entire solution should be affordable and possible to make at a scale so it can be used in rural and semi-urban areas of Pakistan that are prone to disasters.

The Emergency Rescue Node Communication System, at Bahria University Karachi Campus is working on this problem.

1.3 Goal of this Project

The main goals of this project are to help create a system for emergency rescues:

1. We want to make special devices called rescue nodes that can create a Wi-Fi network and a webpage for people to send SOS messages from their phones. This way people in trouble can send emergency alerts without needing to install an app or have internet.
2. We aim to make a way for these rescue nodes to talk to each other using a technology called LoRa. This will help send messages, including the persons location from one node, to another until it reaches a main node called a gateway.
3. We plan to use the phones built-in GPS to find the persons location. This way we don't need to add GPS devices to the rescue nodes but we can still find the person accurately.
4. The gateway node will get the SOS messages show them on a screen and send them to a website where rescue teams can see where help is needed.
5. We also want to make an Android app that lets people send SOS messages manage emergency contacts and share their location with the rescue team.

6. We will test the system to make sure it works properly. This includes checking if the nodes can talk to each other if the webpage works and if the location information is accurate. This will help us know if the system is ready to use in emergencies.

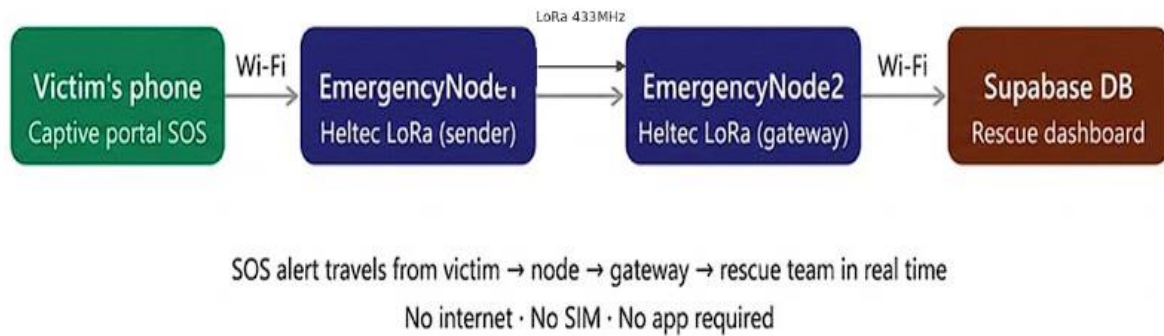


Fig. 1.1 — System Overview Data Flow Diagram

1.4 Scope of the Project

The project is about creating a design for a rescue communication system. This system will have three nodes. Will work offline using LoRa mesh technology.

The project has three parts:

- **Functional Scope:** this is where the system is always on and ready to help. Each node will have a Wi-Fi connection so people in trouble can send an SOS message. The node will also listen for messages on the LoRa channel. Send them to the right place. The person in trouble can use their smartphone to send their location. The message will then be sent through nodes to the main gateway. The gateway will show the message on a screen and send it to the rescue team. There is also an Android app that will give help when connected to a rescue node.
- **Hardware Scope:** we will use three boards that have everything we need on them. These boards have a computer, a LoRa radio, a small screen, Wi-Fi and a battery. We do not need to make any custom parts or use modules. Each node will be powered by a battery.
- **Software Scope:** we will write code in C and C++ to make the system work. This code will make the Wi-Fi connection create a web server send and receive messages and show things on the screen. We will also make an Android app for emergency help. The rescue team will have a web page where they can see all the messages.

Some things are not part of this project like using LoRaWAN or The Things Network making custom parts or using GPS hardware. We are also not making an app, for iOS devices. Deploying the system on a large scale at this time [2].

1.5 SDG Statements

This project helps the United Nations reach some of its Sustainable Development Goals. It does this by giving people a way to communicate during disasters in places that do not have a lot of resources. The Emergency Rescue Node Communication System is very good at helping with SDG 3: Good Health and Well-Being. It lets people who are in trouble send out a distress signal and get help from rescue teams when other ways of communicating are not working. This system helps connect people who are trapped with the people who are trying to rescue them. It saves lives. Helps people who are hurt during disasters. The few hours after a disaster are very important and this system helps people get the help they need during this time[11].

It also helps with SDG 11: Sustainable Cities and Communities. The system gives people a way to communicate during emergencies that's affordable and can be set up quickly. It works when other systems are not working. This helps communities deal with disasters and saves lives. The system is very good at helping communities be prepared for disasters. It keeps working even when other systems are not.

The project also helps with SDG 13: Climate Action. It does this by helping people deal with the effects of climate change. Climate change is causing floods and other disasters in places, like Pakistan and South Asia. The Emergency Rescue Node Communication System is a tool that can help people in these areas. It is a way for communities to adapt to the effects of climate change. It can be used in places where disasters are more likely to happen.

1.6 Document Overview

This report has nine chapters.

Chapter 1: Gives an overview of the project. It talks about the projects background the problem we are trying to solve our goals, what we will cover and how it aligns with the UNs Sustainable Development Goals. It also explains the parts of the project and suggests how to design it.

Chapter 2: Looks at what's out there. It reviews existing emergency communication systems, their limitations and research related to LoRa.

Chapter 3: Analyzes the system. It includes diagrams that show how the system works how data flows and what the system needs to do.

Chapter 4: Discusses design considerations. This includes the hardware and software environment, limitations, strategies for designing the system and how we will develop it.

Chapter 5: Explains the hardware. This includes the LoRa ESP32 V2 platform, how to configure SX1276 integrating phone GPS and the power system.

Chapter 6: Talks about the firmware and mesh communication. This includes the portal, a multi-hop routing algorithm and the packet structure.

Chapter 7: Presents the Android application. It explains the design and features.

Chapter 8: Explains the rescue monitoring dashboard. This includes its architecture and what it can do.

Chapter 9: Concludes the report. It provides testing results evaluates performance and offers conclusions and suggestions, for work.

1.7 Project Modules

The system architecture is partitioned into five primary interacting modules:

Table 1-1: Overview of System Modules

Module	Hardware Components	Software Components
Rescue Node Hardware	LoRa ESP32 V2 (ESP32 + SX1276 built-in), 0.96" OLED, JST LiPo connector, TP4054 charging IC, antenna	Node firmware (Arduino/C++), Wi-Fi AP configuration, OLED display driver
Captive Portal SOS Interface	Victim smartphone (any), LoRa ESP32 V2 Wi-Fi AP	ESP32 WebServer, HTML/CSS/JS captive portal, Browser Geolocation API, HTTP POST handler
LoRa Mesh Communication	SX1276 transceiver (built-in LoRa ESP32 V2), LoRa antenna	RadioLib library, multi-hop routing algorithm, duplicate suppression, collision avoidance
Android Mobile Application	Android smartphone	Android SDK, SOS module, emergency contacts, offline safety guides, location sharing
Rescue Monitoring Dashboard	Web dashboard, Sup abases PostgreSQL	Supabase PostgreSQL , REST API, real-time WebSocket subscriptions, map-based GPS visualization, alert management

1.8 Overview and Design Recommendations

The emergency rescue communication system has three types of nodes that work together using LoRa-based communication.

1.8.1 Rescue Nodes (Node 1 Node 2):

Each Rescue Node is made with an ESP32 that has LoRa inside. It works as a Wi-Fi access point called EmergencyNode. When someone connects to this access point, they are sent to an SOS web page on the ESP32. The person using it puts in information like their name, phone number and what is happening in the emergency. The system also finds out where the person is using the browser [3].

This information is sent to the ESP32 using a HTTP request. The ESP32 then makes it into a LoRa packet. Sends it using a 433 MHz frequency. The node shows if the message is sent or not on a display.

1.8.2 Gateway Node (Node 3):

The Gateway Node listens for LoRa messages all the time. When it gets a SOS message it shows important information like who the person is, their phone number and where they are, on a screen. This way the people helping with the rescue can see it away. The Gateway Node also sends this information to a website where people can track what is happening.

1.8.3 Multi-Hop Communication:

All nodes help send messages to make sure they get through in a disaster. If a Rescue Node cannot send a message directly to the Gateway Node, other nodes help by sending it for them. This makes sure the message gets through even if the infrastructure is damaged.

The emergency rescue communication system is strong. Does not use a lot of power because it uses ESP32-based nodes LoRa communication and a way for nodes to send messages to each other. The emergency rescue communication system is also independent of infrastructure. This means the emergency rescue communication system works even when other systems are not working [3].

1.8.4 Design Recommendations

1. **LoRa ESP32 V2 on Custom PCB:** The LoRa ESP32 V2 combines ESP32, SX1276, OLED, Wi-Fi and battery management on a board. This gets rid of the need to design and make a PCB, which lowers the cost per node and speeds up development. It also results in a hardware unit that's ready to use in the field without any extra assembly.
2. **Phone GPS of Hardware GPS Module:** Getting GPS from a smartphone's browser using the Geolocation API means we don't need a hardware GPS module on every node. This reduces cost and complexity while providing location accuracy than standalone UART GPS modules.
3. **Open Wi-Fi Hotspot:** The hotspots on rescue nodes don't have a password so anyone can connect away in an emergency without needing to share or remember a password.
4. **433 MHz for Pakistan and Asia:** Using the SX1276 at 433 MHz gives obstacle penetration and works without a license. It also provides a range of 800m to over 1 km, which is well suited for disaster terrain.

5. **3.7V LiPo Battery:** The LoRa ESP32 V2 has a built-in charging circuit and JST connector designed for single-cell 3.7V LiPo batteries. These batteries provide 10–2 hours of field operation per charge. We plan to add charging in the future to enable nodes to operate indefinitely.
6. **Scalability:** The mesh architecture grows linearly with new nodes joining without needing to reconfigure existing ones. For disaster deployments we recommend using multiple gateway nodes and placing nodes in high locations to maximize coverage and redundancy. The LoRa ESP32 V2 and mesh architecture make it easy to add nodes. LoRa ESP32 V2 is a solution, for disaster response.

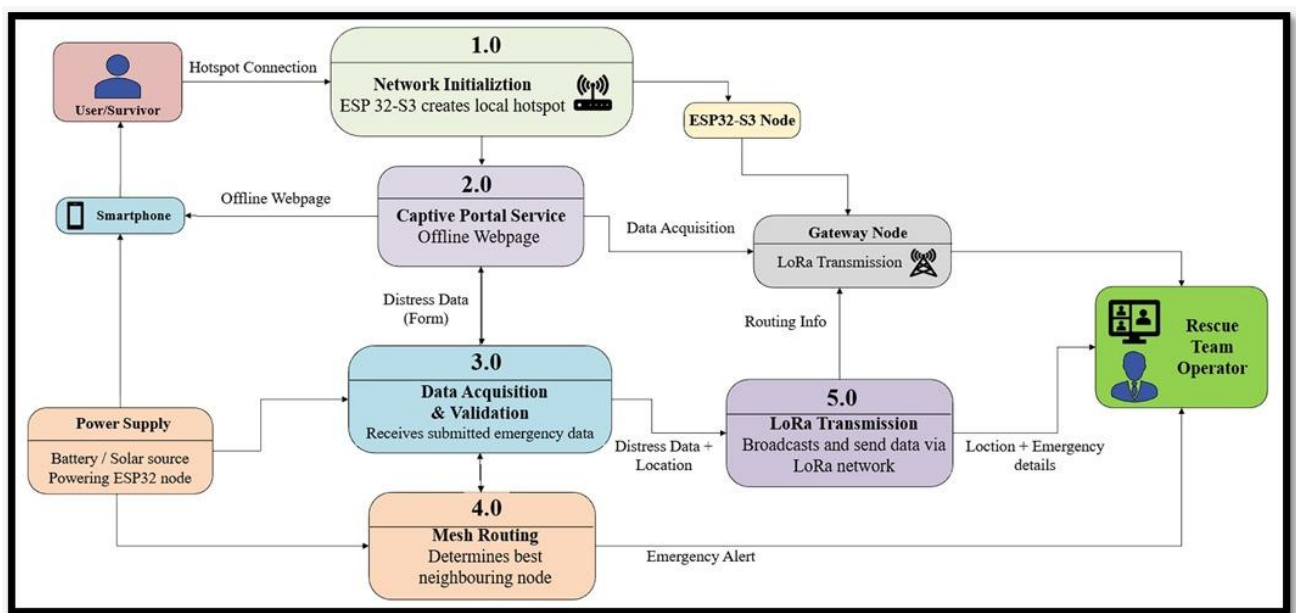


Fig. 1.2 — System Overview Data Flow Diagram

This diagram provides a high-level overview of the five core processes of the Emergency Rescue Node Communication System — from network initialization and captive portal service through data acquisition, mesh routing, and final LoRa transmission to the rescue team coordinator.

CHAPTER 2: BACKGROUND AND LITERATURE REVIEW

2.1 Existing Systems

2.1.1 Existing Systems Description

Emergency communication is really important during disasters. There are four types of technology used for this. Each type has its way of working and its own advantages and limitations. Emergency communication systems that use networks are the most common type used around the world. These systems have features like emergency channels and location-based alerts. Smartphones can also send emergency messages and share location information. However, these systems need base stations, connectivity and network infrastructure to work, which can be damaged or overloaded during disasters. Satellite-based systems are another option. These systems can work when other infrastructure is damaged. They provide coverage around the world and are used by military organizations, humanitarian agencies and governments. Satellite communication devices are expensive need special training and require subscription services making them too costly for individual communities or small emergency response teams in places like Pakistan.

Internet-based emergency applications are becoming more popular. These apps use smartphone GPS and cellular or Wi-Fi connectivity to send emergency alerts. They are useful in areas with working infrastructure. They do not work when internet connectivity is lost which is often the case during big disasters. Low Power Wide Area Network technologies like LoRa are being researched for use in emergency communication. LoRa technology allows data to be sent over distances using very little power. It can be used to create emergency mesh networks that do not need any infrastructure. This makes it really useful for emergency communication in areas where other systems are not working [1].

- LoRa technology is different from LoRaWAN because it does not need network server infrastructure [9].
- It can be used for peer-to-peer communication making it suitable for emergency networks.
- The LoRa ESP32 V2 board used in this project has a transceiver that implements the LoRa layer.
- This allows for trade-offs between range, data rate and power consumption.

Emergency communication during disasters is a challenge. Different technologies have their strengths and weaknesses. It is essential to find the solution for each situation. Emergency communication systems like networks, satellite-based systems, internet-based applications and Low Power Wide Area Network technologies including LoRa all play a crucial role, in saving lives during disasters. Emergency communication systems need to be reliable and work when other infrastructure is damaged. LoRa technology has the potential to provide this kind of reliability [2].

2.1.2 Problems in the Existing Systems

When we look at the emergency communication solutions that're already out there, we find six big problems that together make a strong case for creating a new system.

- **Infrastructure Dependency:** Emergency communication systems that use cell phones and the internet depend on things like towers, power grids and core networks. Natural disasters often destroy these things. So, when we need emergency communication the most these systems fail us.
- **Prohibitive Cost of Infrastructure-Independent Solutions:** Satellite communication systems do not depend on infrastructure. They are very expensive. The cost of the equipment and running it is too high for communities, volunteer rescue groups and disaster response teams in developing areas. A solution that costs thousands of dollars per node is not something that can be used by a community.
- **Network Congestion During Peak Demand:** When a disaster happens the cell phone networks that are still working get overwhelmed by all the people trying to use them at the time. This means that even if the cell phone network is still working it is not reliable when we need it the most.
- **Absence of Offline Location Tracking:** Most of the time emergency communication systems that work offline do not have a way to send location data. This means rescue teams have to search for people without knowing where they are. It takes them longer to find people. They are less likely to find them in big areas.
- **Single Point of Failure in Centralized Architectures:** Emergency communication systems that are centralized have one point that everything depends on. If this point fails the whole system fails. If we use a mesh architecture, we have many different paths for the communication to go through. This makes the system more reliable [1].
- **Absence of Affordable Integrated Solutions:** There is no emergency communication system that's affordable and has all the things we need. We need a system that can work offline send location data and has a display that people can read in the field. We also need a system that has a dashboard, for rescue teams. Existing systems have some of these things but not all of them. They are not affordable.

2.2 Related Work

The Emergency Rescue Node Communication System is based on a lot of research that has been done recently. Petäjäjärvi et al. Did some work on LoRa LPWAN technology. They found out that it works well for distance wireless communication. They were able to send data over fifteen kilometres in areas

and several kilometres in cities with buildings and other things in the way. [8] Their work showed that LoRa is a choice for emergency communication. It has the range and power that we need for the Emergency Rescue Node Communication System. That is why we chose to use the SX1276 transceiver and the 433 MHz operating frequency.

Bor et al. Did some research on how LoRa Low-Power Wide-Area Networks work when there are a lot of devices connected. They found out that LoRa networks work well even when there are devices connected. They also found out that we need to be careful about how we set up the network so that the devices do not interfere with each other.[2] Their findings helped us make decisions about how to design the mesh routing for the Emergency Rescue Node Communication System. For example, we chose to use Spreading Factor 9 and a random back-off collision avoidance in the relay algorithm.

Akyildiz et al. Did some research on wireless mesh network architectures. They found out some things about how to route data in a mesh network. Their work helped us understand how to make the network reliable and how to manage the data. We used their findings to design the suppression mechanism and the maximum hop-count enforcement in the LoRa mesh protocol.

There have been some projects that have used LoRa-based sensor networks for disaster communication. Most of these projects only focus on sending data from device to device. They do not have a way for people to interact with the system or send alerts. They also do not have a way to capture the location of the person who needs help. The Emergency Rescue Node Communication System is different because it has an interface for the person who needs help. It also has a way to capture the location of the person using their phones GPS.

Some people have used portal web interfaces on embedded microcontrollers before. We are using it in a new way for emergency SOS submission. When someone connects to the Wi-Fi network, they are automatically redirected to the SOS web page. The system also uses the Geolocation API to get the persons location. This is a way of doing things that makes it easier for people to send alerts.

Research has shown that the ESP32-based Wi-Fi access point architecture is very responsive. It can handle HTTP requests in than one second. This makes it perfect for emergency situations. The RadioLib library that we are using for the SX1276 LoRa control has been tested before. It works well for sending and receiving LoRa packets.

One of the things about the Emergency Rescue Node Communication System is that it uses the browser-based Geolocation API to get the persons location. This means that we do not need to use a GPS module. This makes the system less expensive and less complicated. Modern smartphones are very good at getting the location. They can get the location within 3-5 meters in areas. This is better, than the 2.5-5-meter CEP that standalone GPS modules can get. It eliminates the extra cost and complexity of the GPS module.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Work Analysis

The Emergency Rescue Node Communication System is made to be an independent system that always has Wi-Fi hotspots that people in trouble can use. It takes in distress messages. Sends them to other parts of the network without needing anyone to help at each stop. The system makes sure messages get through and that people in trouble can use it without needing the internet cell phones or power from the grid.

When the system starts up each part of the network does a thing. It gets the special radio ready to work at 433 MHz starts the Wi-Fi begins the web server and shows what is going on a screen. Then it keeps doing two things at the time: it helps phones that are connected and it listens for new messages on the network.

There are two ways that each part of the network works depending on what it is supposed to do.

Rescue Node Mode is for the two parts of the network. They keep the Wi-Fi open so people can connect. When someone does, they are sent to a page where they can send a distress message. The person types in their name, phone number. The phone automatically sends its location. The message is then sent to parts of the network.

Gateway Node Mode is for the part of the network. It listens for messages. Shows the important information on its screen. It also sends the information to a website where people can see what is happening and plan a rescue.

In both scenarios, each node in the network keeps a local record of previously processed messages and uses deduplication logic to ensure that no message is retransmitted redundantly. In addition, each packet is controlled by a programmable time-to-live (TTL) timer that decrements with each hop, preventing messages from being circulated indefinitely throughout the mesh and reducing network congestion.

The Emergency Rescue Node Communication System runs continuously and autonomously, providing continuous assistance to individuals in distress even in the most severe infrastructure-degraded disaster scenarios. By utilizing the long-range, low-power LoRa mesh network for SOS packet propagation, the system assures dependable message transmission among geographically distributed nodes without relying on traditional cellular or internet infrastructure. This makes the Emergency Rescue Node Communication System an essential tool for rescue teams.

3.1.1 Work Flow Diagram

The system workflow follows a structured sequence of states from node initialization through continuous autonomous rescue operation. Two workflow diagrams capture the complete system behaviour:

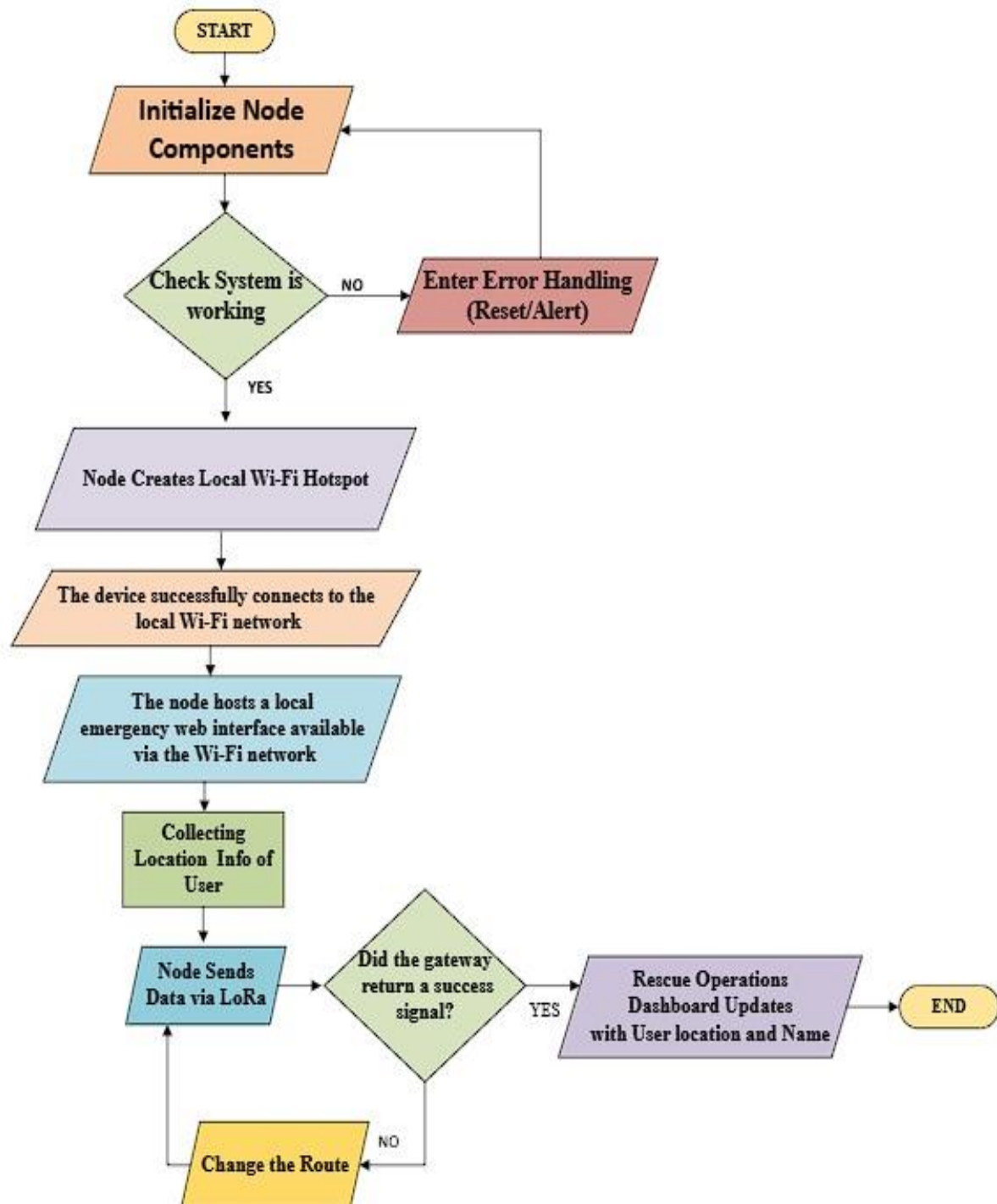


Fig. 3.1 — Over all System Workflow Diagram

3.1.2 Work Breakdown Structure

Table 3-1: Work Breakdown Structure

Phase	Description of Activities	Status
P1	Project Planning: Literature review, problem identification, LoRa mesh feasibility analysis, scope definition, and team role assignment.	Completed
P2	Hardware Selection and Procurement: LoRa ESP32 V2 board selection, LiPo battery procurement, antenna selection, and component verification.	Completed
P3	Node Firmware Development: ESP32 Wi-Fi AP configuration, ESP32 Web Server captive portal implementation, HTTP POST SOS handler, and Laptop display integration.	Completed
P4	LoRa Mesh Communication: SX1276 Radio-Lib integration, SOS Packet structure definition, multi-hop routing algorithm, duplicate suppression, and collision avoidance implementation.	Completed
P5	Phone GPS Integration: Browser Geolocation API invocation in captive portal HTML, GPS coordinate extraction from HTTP POST, and coordinate embedding in LoRa packet.	Completed
P6	Gateway and Dashboard: Gateway node LoRa receive firmware, serial forwarding to dashboard, web dashboard development, map-based GPS visualization, and alert management.	Completed
P7	Android Application Development: SOS module, emergency contacts, pre-loaded helplines, offline safety guides, and live location sharing via node hotspot.	Completed
P8	System Integration and Testing: End-to-end three-node communication testing, captive portal accessibility testing on multiple smartphone models, GPS accuracy validation, and dashboard alert delivery verification.	Completed

3.2 Data Analysis

The Emergency Rescue Node Communication System relies on a trustworthy flow of information. This information comes from two places: the SOS forms that people in trouble fill out and send to the rescue node web server and the LoRa packet data that the relay and gateway nodes receive over the radio. It is really important that this information is handled correctly every step of the way because the Emergency Rescue Node Communication System is used in life-or-death situations. If an SOS message gets lost or messed up it could mean that someone does not get rescued.

When the information gets to a node the system checks it to make sure it is correct and makes sense. It takes the GPS coordinates from the SOS form. Converts them into a format that the system can understand. It also checks to make sure these coordinates are, within a range. Each message is given an identifier so that the system can tell if it has seen the message before. This helps prevent the system from sending out alerts. At the relay level the system checks the LoRa packets it receives to see if it has already seen them. If it has the system does not send out another alert. The Emergency Rescue Node Communication System keeps track of all the SOS alerts it has received which helps the rescue team understand what happened see where the nodes are and figure out how well they responded to emergencies over time. The Emergency Rescue Node Communication System uses this information to improve how it works [6].

3.2.1 Data Flow Diagram

The Data Flow Diagrams below illustrate how data travels through the system from physical environmental parameters to cloud-based visualization.

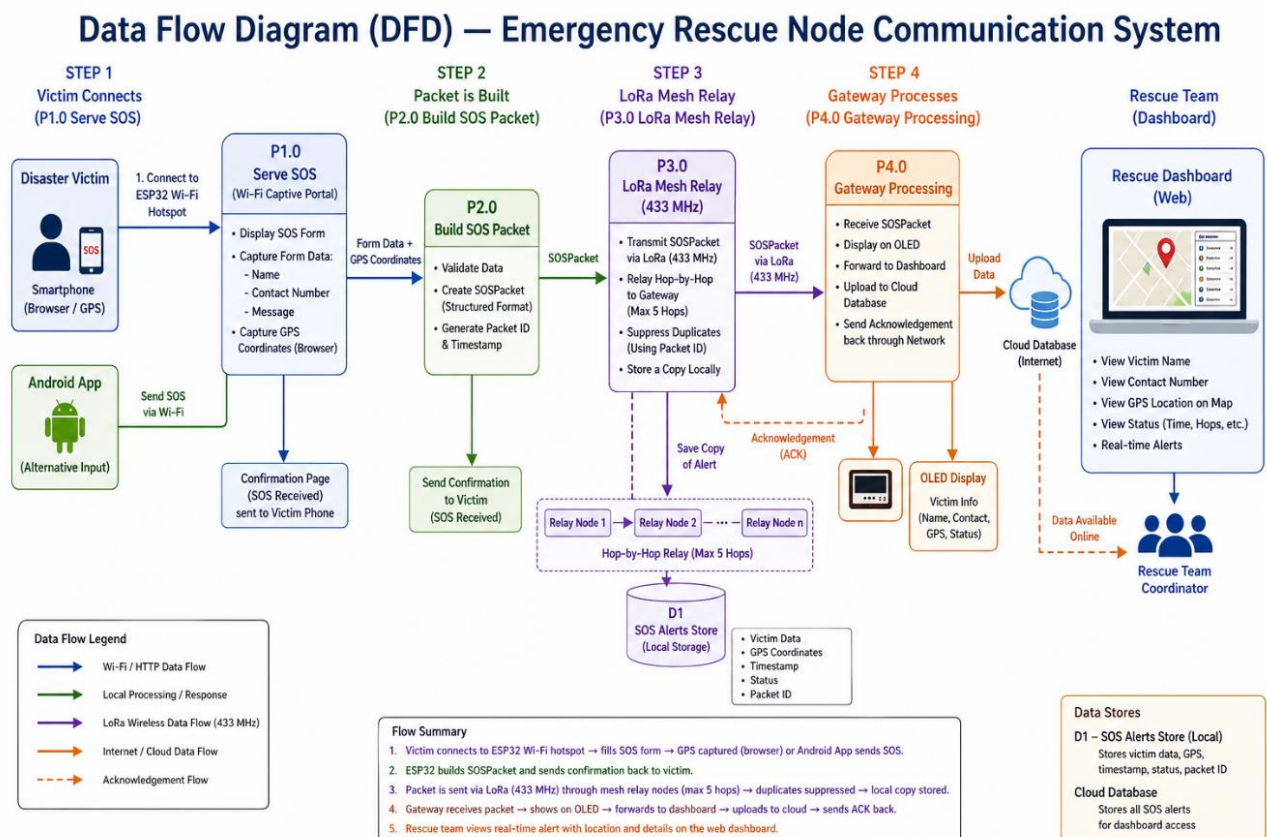


Fig. 3.2 — Data Flow Diagram

3.3 Requirements of the System

3.3.1 Clients, Customer and Users

The people who use the Emergency Rescue Node Communication System are disaster victims. These are people who have been affected by things like floods, earthquakes and other natural disasters. They have lost access to all the ways of communicating with others. These people will probably only know how to use a smartphone in a basic way. They may be under a lot of stress. They may be in a place with very little light. Their phone batteries may be running out. They may not know anything about the rescue node system.

The Emergency Rescue Node Communication System is designed to be very easy to use. The people who are using it only have to do a thing: connect to Wi-Fi fill out a simple form and press send. They do not have to install any applications register for anything or remember a password. They do not even need to have an internet connection.

There are people who use the Emergency Rescue Node Communication System too. These are the rescue team coordinators and the emergency responders who are in the field. They use the rescue dashboard. They read the SOS alerts that come in on the gateway node Laptop screen. These people need to be able to see the alerts quickly and clearly. They need to know where the emergency is so they can send help right away. The dashboard is designed to be very easy to read and understand. It shows the alerts in the order they were received and it shows the location on a map.

There are also people who are involved with the Emergency Rescue Node Communication System. These are the people, in charge of managing disasters and the people who are supervising the project. They look at how the system is working and they check to see how many people it can help. They also look at how easy it's to set up the system in different places. They do all of this when they are testing the system and showing it to others.

3.3.2 Resource Requirements

To make this system work we need some hardware and software.

The system needs the following hardware and software to work properly.

Here are the software resources we need:

- **Computational and Communication:** We need three LoRa ESP32 V2 development boards. Each board has an ESP32 core 240 MHz microcontroller, an SX1276 LoRa transceiver, a 0.96-inch SSD1306 OLED display, Wi-Fi 802.11 b/g/n and a TP4054 LiPo charging circuit all on one unit.

- **Power:** We need three single-cell 3.7V LiPo batteries. These batteries should be between 400–1000 mAh. They should also have JST 1.25mm 2-pin connectors that can plug into the LoRa ESP32 V2 built-in charging port [5].
- **Antenna:** Each node needs a LoRa 433 MHz antenna to get the communication range.
- **Victim Interface:** We can use any Android or iOS smartphone that has GPS and a web browser. We do not need any hardware for this.
- **Gateway Output:** We need a PC or laptop that can connect to the gateway node using a USB connection to get the dashboard data. We can also use a Wi-Fi network to send the dashboard data wirelessly.

Here are the software resources we need:

- **Development Environment:** We will use Arduino IDE 2.x with the ESP32 board support package to develop the firmware.
- **LoRa Library:** We will use Radio Lib to control the SX1276 LoRa transceiver send packets and receive packets.[9]
- **Web Server:** We will use the ESP32 Arduino Web Server library to host a portal HTTP server.
- **Android Application:** We will use the Android SDK with Java or Kotlin to develop the application.
- **Dashboard:** We will use web technologies, like HTML, CSS and JavaScript and a map integration library to show the GPS data.
- **Version Control:** We will use Git and GitHub to work as a team and manage the different versions of the firmware.

3.3.3 Data Requirements

The system needs to handle and send types of information through the rescue communication pipeline. The data is categorized into three main types based on the communication stages.

SOS Submission Data (from Victim to Node via HTTP POST):

This data is submitted by the victim through a web interface and includes the following fields:

- Victims name: a text, up to 30 characters.
- Contact number: a text, up to 15 characters.
- Emergency description: a text, up to 60 characters.
- GPS latitude: a number between -90.0 and 90.0 degrees.
- GPS longitude: a number between -180.0 and 180.0 degrees.

LoRa Packet Data (from Node to Node via 433 MHz Radio):

This data is transmitted between nodes over LoRa communication and contains routing and identification information:

- Sender node ID: a number, 8 bits that identifies the node that sent it.
- Target node ID: a small number, 8 bits that identifies the node it is meant for (255 is for everyone).
- Hop count: a number, 8 bits that counts how many nodes it has passed through maximum 5.
- Packet ID: a unique number, 16 bits for each node that sends it to avoid duplicates.
- Latitude and Longitude: numbers that show where it is.
- Message payload: a message, 60 characters that has the victim's name, contact and description.

Dashboard Alert Data (from Gateway to Web Dashboard):

All SOS alerts are saved in a database with the following details:

- AlertID
- Timestamp
- Victim Name
- Contact
- Latitude
- Longitude
- SourceNodeID
- Hop Count
- Status which can be NEW, ACKNOWLEDGED or RESOLVED.

3.3.4 Non-Functional Requirements

Table 3-2: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Reliability	LoRa packet delivery must succeed across at least two hops in the three-node configuration under normal deployment conditions. Duplicate suppression must prevent any SOS alert appearing more than once on the rescue dashboard.

ID	Category	Requirement
NFR-02	Performance	End-to-end SOS delivery from HTTP POST submission to gateway Laptop display must complete within 10 seconds under normal operating conditions.
NFR-03	Usability	The captive portal SOS interface must be operable by a non-technical user in under 60 seconds from Wi-Fi connection to SOS transmission confirmation, requiring no prior training or app installation.
NFR-04	Availability	Each rescue node must operate continuously for a minimum of 10 hours on a fully charged 400 mAh LiPo battery without interruption to Wi-Fi hotspot or LoRa listening functions.
NFR-05	Portability	The complete rescue node hardware must be deployable in any outdoor or indoor disaster environment without any supporting infrastructure, power grid access, or network connectivity.
NFR-06	Maintainability	Node firmware must be updatable via USB connection without hardware modification. Node ID and LoRa frequency must be configurable via a single constant definition before compilation.

Table 3-3: Functional Requirements

ID	Requirement	Description
FR-01	Wi-Fi Hotspot	Each rescue node shall create and maintain an open Wi-Fi access point continuously from boot.
FR-02	Captive Portal Redirect	Node shall redirect all HTTP requests from connected devices to the SOS submission page automatically.
FR-03	Phone GPS Capture	The SOS page shall invoke the browser Geolocation API to capture victim smartphone GPS coordinates without manual entry.
FR-04	SOS Transmission	Node shall construct and transmit a structured LoRa SOS packet within 3 seconds of receiving a valid HTTP POST submission.
FR-05	Multi-hop Relay	Relay nodes shall forward received LoRa packets not addressed to themselves, provided hop count is below maximum threshold.
FR-06	Duplicate Suppression	All nodes shall maintain a seen-packet buffer and discard any packet whose ID has been previously processed.

ID	Requirement	Description
FR-07	Status Display	All nodes shall display current operational status, last received or transmitted SOS summary, and GPS coordinates [9].
FR-08	Gateway Forwarding	The gateway node shall forward received SOS data to the rescue monitoring dashboard upon packet receipt.
FR-09	Dashboard Alert Display	The rescue dashboard shall display incoming SOS alerts with victim name, contact, GPS coordinates, timestamp, and source node within 5 seconds of gateway receipt.
FR-10	Android SOS	The Android application shall provide a one-tap SOS button that submits an emergency alert via the rescue node hotspot when in range.

3.4 Proposed Solution

The final solution is a communication system. It helps people in trouble send messages to rescue teams. The system uses a kind of network called LoRa. It works without internet, phones or electricity. It has three parts: two small devices that people can use to send messages and one device that connects to a big screen for the rescue team.

Each small device can do two things at the time. It can talk to people phones using Wi-Fi. It can also send messages to devices using LoRa. The system gets location information from people phones. It uses a tool called Geolocation API. This way the devices do not need GPS hardware. The system sends all information, like where the person's what they need in a small package. This package is designed to be sent efficiently.

The device that connects to the screen can do two things. It can show messages on the screen. It can also send information to a website that the rescue team uses. On the website the rescue team can see where people are on a map. They can use this to decide where to send help.

This system works on its own. It does not need people to make it work. If one device breaks the others can still work. This way the system can keep helping people even if something goes wrong. The LoRa network is very good, at keeping working even if some devices fail. If one device stops working the others can still talk to each other. This means that the system can keep sending messages even if some devices are not working.

The rescue team can use this system to help people easily. They can see where people are. Send help to the right place. The system is designed to be easy to use and reliable.

CHAPTER 4: SYSTEM DESIGN

4.1 Project Modules

The Emergency Rescue Node Communication System is made up of five parts that work together to handle the physical, communication and interface-based operations.

1. The Wi-Fi Access and Portal Module is in charge of creating and maintaining the Wi-Fi access point on each LoRa ESP32 V2 node. It also hosts the portal SOS web interface directly on the ESP32 Web Server. This module looks at all HTTP traffic from victim smartphones and redirects it to the SOS submission page. This means that any victim with a smartphone can send an emergency alert without having to install an app.
2. The GPS Capture and Packet Construction Module handles a thing. It gets the location of the victim's smartphone using the browser Geolocation API. It then takes the latitude and longitude coordinates from the HTTP POST submission and checks if the GPS data is correct. After that it puts together a SOS-Packet that has all the victim's details, phone-sourced GPS coordinates routing metadata and a unique packet identifier to prevent duplicate packets.
3. The LoRa Mesh Communication Module is responsible for all communication between nodes using the SX1276 LoRa transceiver on each LoRa ESP32 V2 board. This module sends SOS-Packets at 433 MHz Handles multi-hop relay routing with hop-count enforcement. It also prevents collisions using back-off and detects duplicate packets using a circular seen-packet buffer [9].
4. The Gateway and Dashboard Module works on the designated gateway node, which's Node 3. It receives incoming LoRa SOS packets. Shows victim details and GPS coordinates on the built-in laptop screen. This allows rescue teams to read the information directly in the field. It also sends structured alert data to the web-based rescue monitoring dashboard for emergency coordination.
5. The Mobile Application Module is the companion Android application. It lets users send SOS alerts via rescue node hotspot and manage emergency contacts. It also has pre- Pakistan emergency helplines, offline first-aid and safety guides and live location sharing when connected to a rescue node. The Emergency Rescue Node Communication System and its modules work together to help people in emergency situations. The Emergency Rescue Node Communication System is a tool, for rescue teams and victims alike.

4.2 Design Constraints

The Emergency Rescue Node Communication System was designed with a lot of things to consider. The people who made it had to think about the operational and economic parts that would affect the whole system.

When it comes to radio communication the LoRa SX1276 transceiver can only do one thing at a time. It can. Send or receive information but it cannot do both at the same time. This means that when a node is sending an SOS message it cannot hear any LoRa messages at that time. To fix this problem the system waits for an amount of time before sending the message again. This helps prevent nodes from sending messages at the same time and causing problems.

The LoRa ESP32 V2 was also designed to work with a type of battery. It needs a 3.7V LiPo battery to work. It cannot use 5V or 3.3V batteries. The system uses a lot of power so it needs a battery to work for a long time. The battery has to be at 400 mAh to make sure the system can work for many hours.

The Emergency Rescue Node Communication System also uses GPS to find the location of the person who needs help... It does not have a GPS device built in. Instead, it uses the GPS on the persons smartphone to find their location. This means that the person has to have a smartphone with GPS and have it turned on. If they do not have a GPS-enabled smartphone or if they have turned off the location services the system will not be able to find their location. The system is designed to handle this situation. It can still accept the SOS message. Show a message on the dashboard that says the location is not available [4].

The people who made the Emergency Rescue Node Communication System also had to think about the cost. They wanted to make sure that the system was affordable and could be used in different places especially in areas that are prone, to disasters. That is why they used parts that're already available and not too expensive. The LoRa ESP32 V2 is an example of this. It has different parts built into one unit, which makes it cheaper and easier to use. This means that the system can be made and used without spending a lot of money on custom parts.

4.3 Hardware and Software Environment

4.3.1 Hardware

The hardware ecosystem is based on three identical LoRa ESP32 V2 development boards, each acting as a self-contained rescue node:

- **LoRa ESP32 V2 Development Board (×3):** Each board has an ESP32 microcontroller, LoRa transceiver, laptop display, Wi-Fi, Bluetooth and a battery charging circuit. These boards are self-contained so you do not need any extra parts.
- **LoRa Transceiver (built into LoRa ESP32 V2):** Operates at 433 MHz and communicates with the ESP32 microcontroller using SPI. The configuration settings include Spreading Factor 9, 125 kHz bandwidth, coding rate 4/7 and 17 dBm output power.
- **Power System:** Each node uses a 3.7V LiPo battery connected to the LoRa ESP32 V2 board through a built-in connector. A TP4056 USB Type-C charging module is also added for

recharging, which protects the battery from overcharging and over-discharging. The LoRa ESP32 V2 board powers up directly from the battery through its connector [5].

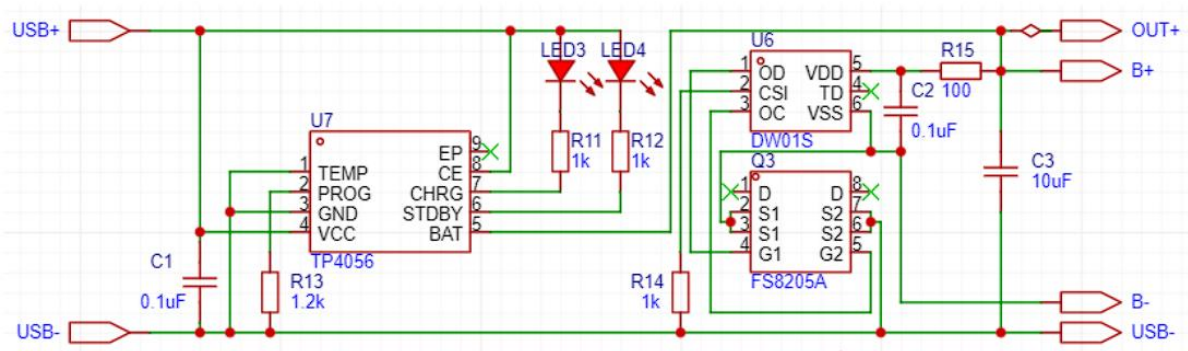


Fig 4.1 - TP4056 charging circuit image

- **433 MHz LoRa Antenna (×3):** An external antenna connects to the LoRa ESP32 V2 board to improve communication range.
- **Victim Smartphone:** Any smartphone with GPS and a web browser works. No extra hardware is needed to track the location.
- **Gateway Laptop:** This laptop connects to Node 3 via USB or local Wi-Fi to get data from the dashboard

Table 4-1: Hardware Components List

Component	Specification / Model	Quantity
Rescue Node Board	LoRa ESP32 V2 (ESP32 + SX1276 + OLED + Wi-Fi + LiPo charging)	3
LoRa Transceiver	SX1276 — built into LoRa ESP32 V2, 433 MHz	3 (built-in)
OLED Display	0.96" SSD1306 128×64 — built into LoRa ESP32 V2	3 (built-in)
LiPo Battery	3.7V, 400–1000 mAh, JST 1.25mm 2-pin	3
LiPo Charging Module	TP4056 with USB Type-C input, 1A charge current, overcharge protection	3
LoRa Antenna	433 MHz, IPEX/SMA connector	3

Component	Specification / Model	Quantity
Victim Interface	Any GPS-enabled smartphone with browser	Per Victim
Gateway Output Device	Laptop with USB serial or Wi-Fi	1
USB Cables	Micro-USB for programming and charging	3
Jumper Wires	Standard prototyping wires	As required

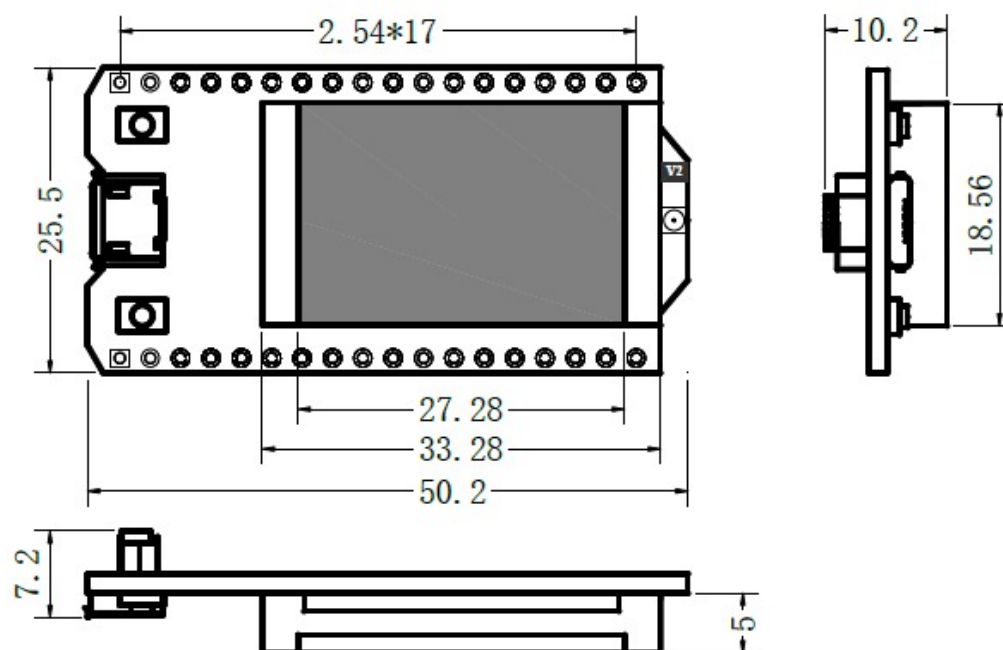


Fig. 4.2: Board Mechanical Dimensions (mm)

4.3.2 Software

- Embedded Firmware:** I wrote the code for this in C/C++ using the Arduino IDE 2.x and the ESP32 Arduino board support package. This firmware does not stop to wait for things to happen. It uses a kind of timing to do many things at the same time. It can handle the Wi-Fi access point and the web server. Send and receive LoRa packets and get GPS data and update the display all without slowing down.

- **LoRa Library:** I used the RadioLib library to control the SX1276 LoRa transceiver. This library helps me send and receive packets and read the signal strength and manage the settings on the ESP32 platform.
- **Display Library:** I used the U8g2lib library to show things on the OLED display. This library helps me show the status of the node and if a message was sent or received and if there is an SOS alert.
- **Web Server:** I used the ESP32 Arduino WebServer library to make a web server. This web server can handle requests from the internet. Send people to the right page. It also helps with the portal.
- **Android Application:** I made a companion app, for Android using the Android SDK and Java/Kotlin. This app lets people send SOS messages and store emergency contacts and safety guides and share their location.
- **Rescue Dashboard:** I made a website using HTML, CSS and JavaScript that shows maps. This website gets the GPS coordinates from the gateway node. Shows where the SOS messages are coming from.
- **Version Control:** I used Git and GitHub to work with my team and keep track of the versions of the firmware and to store the documents.

4.4 Architecture Planning Techniques

The Emergency Rescue Node Communication System was designed to be very flexible. This system is made up of different parts that work together. We wanted to make sure that each part of the system does its job without interfering with the other parts. So, we have the part that talks to the people who need help the part that helps different nodes talk to each other the part that handles all the information that is coming in and the part that keeps an eye on the rescue operation. The people who built this system used a lot of methods to make it work [11].

4.4.1 Algorithm to be used

The LoRa ESP32 V2 system is smart because of two algorithms that are part of its firmware.

Multi-Hop Mesh Routing Algorithm: this algorithm makes sure that LoRa packets are sent to the place in the mesh network. When a node gets a LoRa packet the firmware does the following: it checks the packet ID to see if it has been seen before. If it has the packet is thrown away. If the packet is meant for the node itself it is processed there. If the packet is not meant for the node and it has not been sent many times the node waits for a little while and then sends the packet again. This wait time is random between 50ms and 200ms so that lots of nodes do not send packets at the time. If the packet has been sent many times it is thrown away so that it does not keep going around the network forever.

Phone GPS Capture Algorithm: this algorithm gets the location of the LoRa ESP32 V2 system from a smartphone. It does this by using the phones GPS through a webpage. The webpage asks the phone for its location. When it gets it, the location is put into a form that is sent back to the LoRa ESP32 V2 system. The LoRa ESP32 V2 system then takes this location. Converts it into a special format that can be sent over the LoRa network. It checks to make sure the location is valid and then it sends the location over the LoRa network. The LoRa ESP32 V2 system checks that the latitude is between -90.0 and +90.0 and the longitude is, between -180.0 and +180.0. The LoRa ESP32 V2 system does all of this to make sure that the location of the LoRa ESP32 V2 system is sent correctly over the LoRa network [4][2].

4.4.2 Development Method

When it came to the hardware, we had to do things in an order. We chose the LoRa ESP32 V2 board got the antenna picked a battery and put the physical node together. We did this part in a sequence because changing the hardware after it is put together can be very disruptive and expensive.

The firmware development for the ESP32 nodes the Android application and the rescue dashboard was done in a way. We used an iterative approach. This meant we could quickly make prototypes of parts test the whole communication pipeline and make small improvements to the captive portal interface and routing algorithm based on the results of testing the real hardware. We did this over and over to get the results, for the ESP32 nodes the Android application and the rescue dashboard.

4.5 Project Management Strategies

The project was planned step, by step. We had team meetings every week. We also identified problems that might occur and fixed them before they happened.

We worked on the hardware and software at the time. We checked if they worked well together often. This was done to make sure the node firmware, Android app and rescue dashboard were all compatible.

We made sure to integrate and test these components regularly. This helped us catch any issues on. The node firmware, Android application and rescue dashboard were checked at each integration checkpoint.

The project management approach worked well. We were able to deliver the project on time. The hardware and software worked well together. The node firmware, Android application and rescue dashboard were all compatible.

4.5.1 Task Calendar

The project was tracked through a formal meeting log book maintained throughout the development lifecycle. Key milestones and task completion dates are summarized below:

Table 4-2: Monthly Development Phases and Deliverables

Month	Key Activities	Outcome
September 2025	Literature review, problem definition, LoRa feasibility analysis, project proposal defence.	Project scope approved.
October 2025	LoRa ESP32 V2 board procurement, antenna and battery sourcing, initial firmware setup.	All hardware acquired.
November 2025	Wi-Fi AP configuration, captive portal development, HTTP POST SOS handler implementation.	Rescue node portal functional.
December 2025	SX1276 LoRa configuration, SOSPacket structure design, single-node TX/RX testing.	LoRa communication verified
January 2026	Multi-hop relay algorithm, duplicate suppression, three-node mesh testing, phone GPS integration.	End-to-end mesh functional.
February 2026	Gateway Laptop display, dashboard development, Android app development, mid-year evaluation.	Mid-year evaluation passed.
March–April 2026	Full system integration testing, performance evaluation, documentation, final report preparation.	Final submission ready.

4.5.2 Project Timeline

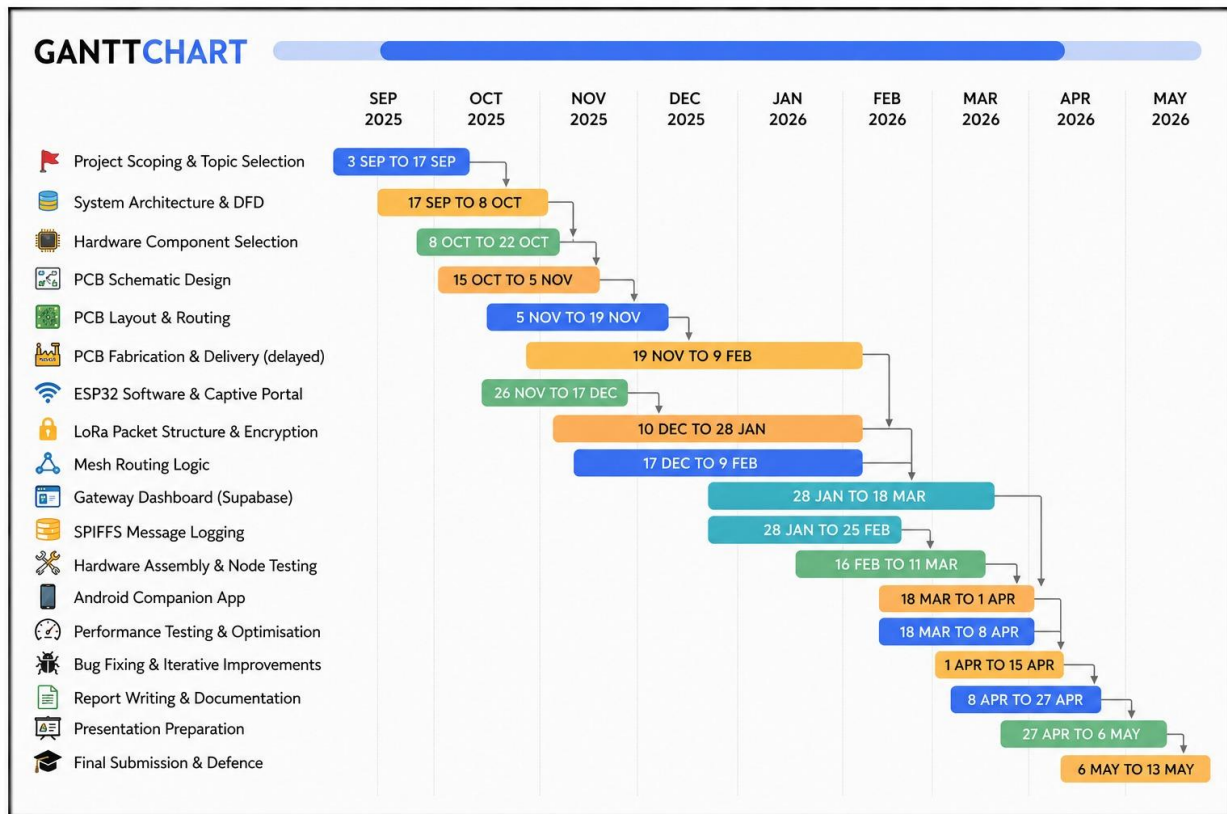


Fig. 4.3 — Project Gantt Chart

Table 4-3: Project Development Phases

Milestone Phase	Primary Objectives	Status
Phase 1: Initiation	Problem definition, literature review, LoRa mesh feasibility analysis, formal project proposal and defence.	Completed
Phase 2: Hardware Procurement	LoRa ESP32 V2 board selection, antenna and LiPo battery procurement, component verification.	Completed
Phase 3: Node Firmware	Wi-Fi AP, captive portal WebServer, HTTP POST SOS handler, OLED display driver.	Completed
Phase 4: LoRa Mesh Communication	SX1276 RadioLib integration, SOS-Packet design, TX/RX testing, multi-hop routing algorithm.	Completed
Phase 5: Phone GPS Integration	Browser Geolocation API in HTML, GPS coordinate extraction from POST, packet embedding.	Completed
Phase 6: Gateway and Dashboard	Gateway receive firmware, serial forwarding, web dashboard, map-based GPS visualization.	Completed

Milestone Phase	Primary Objectives	Status
Phase 7: Android Application	SOS module, emergency contacts, helplines, safety guides, live location sharing.	Completed
Phase 8: Validation and Documentation	End-to-end three-node testing, captive portal accessibility, GPS accuracy, final report.	Completed

4.5.3 Management of Quality

We made sure that quality assurance was a part of every step of the engineering process. We checked how well the LoRa communication worked by testing how packets were delivered between the nodes. We tried all three ways the nodes can talk to each other. From Node 1 to Node 3 using Node 2 to help and sending to all of them at the same time. We looked at how the packets were delivered the signal strength and how long it took for them to get from one end to the other. We did these tests at distances to see how it worked.

We also tested the portal on many different types of Android and iOS phones and browser versions. We wanted to make sure that the portal would always take people to the place and get their GPS location automatically. To check if the GPS was working correctly, we compared the location it gave us to the location. The GPS was usually right within 3 to 5 meters when we were outside with a view of the sky.

We were careful, with the electricity to make sure everything was safe. We only used the built-in power supply that came with the LoRa ESP32 V2 for all the nodes. This way the parts of the circuit that thought and controlled things were never directly connected to the parts that used a lot of power. This kept everything and working well. We used the LoRa ESP32 V2 and the LoRa communication to make sure the quality assurance of the system was good [8].

4.5.4 Human Resource Management

The project was done by three students who studied Computer Engineering at Bahria University, Karachi Campus. They worked together under the supervision of Engr. Usra Sami. Each student had their jobs to do so they could use their skills well:

- Muhammad Tayyab did the hardware setup put together the LoRa ESP32 V2 node, worked on the SX1276 LoRa firmware made a plan for the -hop mesh routing algorithm and tested the system.
- Muhammad Talha worked on the ESP32 Wi-Fi setup made a website for the portal created a handler for SOS messages connected the phones GPS and worked on the gateway firmware.

- Samra Ali made an Android app created a dashboard to watch the rescue designed the user interface and wrote down what they did for the project.

The team met every week to talk about what they had done fix any problems and update their schedule. They met with the project supervisor every week to get help, with the technical parts and to hear what they thought about the project. They talked to the project supervisor, Engr. Usra Sami, to get feedback on the Computer Engineering project.

4.5.5 Risk Management

The project had some risks that we found and fixed before they became problems.

- The LoRa Packet Collision risk was fixed by making the system wait a while before sending messages. This wait time is random. Is between 50 and 200 milliseconds. This helps make sure that two nodes are not sending messages on the channel at the same time on the 433 MHz channel.
- The GPS Unavailability risk was fixed by making the system work even when the GPS is not available. The system can still get all the information from the user and show it on the rescue dashboard. The rescue dashboard will show that the location is not available. It will still get all the other details.
- The Battery Depletion in Field risk was fixed by using batteries that can last for over 6 hours. We also added a charging module to each node so that they can be charged quickly from any power source.
- The Node Failure in Mesh risk was fixed by making the system work even if one of the nodes fails. If one node fails the other nodes can still talk to each other if they are close enough. We can also add nodes to the system without having to change the existing nodes.
- The Portal Not Triggering risk was fixed by making a special handler, for unknown websites. This handler works with both Android and iOS operating systems. Helps the captive portal to work properly.

4.6 Data Conversions

The process of changing data into structured LoRa packet transmission needs precise data conversion at many stages across the system pipeline.

To start with we need to convert the GPS string to a float: the browser Geolocation API gives us latitude and longitude as floating-point numbers with up to 15 places. These numbers are sent as string form fields in the HTTP POST body. The ESP32 WebServer handler gets these strings. Converts them to 32-bit IEEE 754 floats. It checks the values to make sure they are within the range before putting them in the SOS-Packet structure. The 32-bit float representation gives us 5 decimal places of precision which is good enough to locate the victim within 1 meter [8].

Next, we have SOS-Packet serialization: the SOS-Packet C++ struct, which has fields like senderID, targetID, hopCount, packetID, latitude, longitude and message is changed into a byte array and sent as bytes via RadioLib radio transmission function. The node that receives this packet changes the byte array back into the SOS-Packet struct. We get all the fields with the correct type alignment.

Then there is the laptop display: the float GPS coordinates are changed into display strings using a function called dtostrf() for the ESP32. This formats the latitude and longitude to 5 places so it can be shown on the laptop display. For example, it might show 24.86450N, 67.01040E. This fits within the 128×64-pixel Laptop resolution [2].

Finally, there is ID generation: each node has a 16-bit unsigned integer counter that starts at zero when the node boots up. Every time a new SOS submission is made this counter goes up by one creating a packetID for each node. The combination of the senderID and packetID gives us a packet identifier across the whole network, which is good enough for the 50-entry duplicate suppression buffer. The LoRa packet transmission process relies on these LoRa packets. The LoRa packets are what make the whole system work. The LoRa packet transmission is important, for the system pipeline and the system pipeline is what makes the LoRa packet transmission possible.

4.7 Application Program Interface

The system has two ways to communicate: it uses a HTTP interface to talk to smartphones that need help and rescue nodes and it uses a serial or Wi-Fi interface to talk to the rescue monitoring dashboard from the gateway node.

Each rescue node has a web server that can do three things. When you go to the page it shows a special HTML page that has a button to send a distress signal and this page uses JavaScript to figure out where you are. When you send a distress signal the server gets the message. Uses it to make a special packet of information that includes your name, phone number, what is happening and where you are and then it sends this packet using a special kind of radio called LoRa. The server also has two things it can do: it can redirect you to another page if you are using an Android or iOS device so it can figure out if you are connected to the internet.

The gateway node takes the distress signal information it gets. Sends it to the rescue dashboard, which is, like a control centre. It sends this information in a format called JSON and it can do this in two ways: it can send it directly using a wire or it can send it over the internet if it has a connection.

4.8 User Interface

4.8.1 Mobile Application

Table 4-4: Mobile Application Feature Summary

Module	Features
Home Screen	Quick access to all features, system status indicator, emergency SOS button
SOS Alert	One-tap SOS button, automatic phone GPS capture, submits alert via rescue node hotspot
Emergency Monitoring	Displays received SOS alerts, victim locations on map, alert status tracking
Emergency Contacts	Add and manage personal emergency contacts, quick dial during disaster
Emergency Helplines	Pre-loaded Pakistan numbers: Rescue 1122, Police 15, Edhi Foundation 115, Aman Ambulance 1021
Emergency Guide	Offline first-aid instructions, flood survival tips, earthquake response guide
Live Location	Shares user real-time GPS location via node hotspot without internet
User Guide	How to connect to rescue nodes, how to submit SOS, system usage instructions

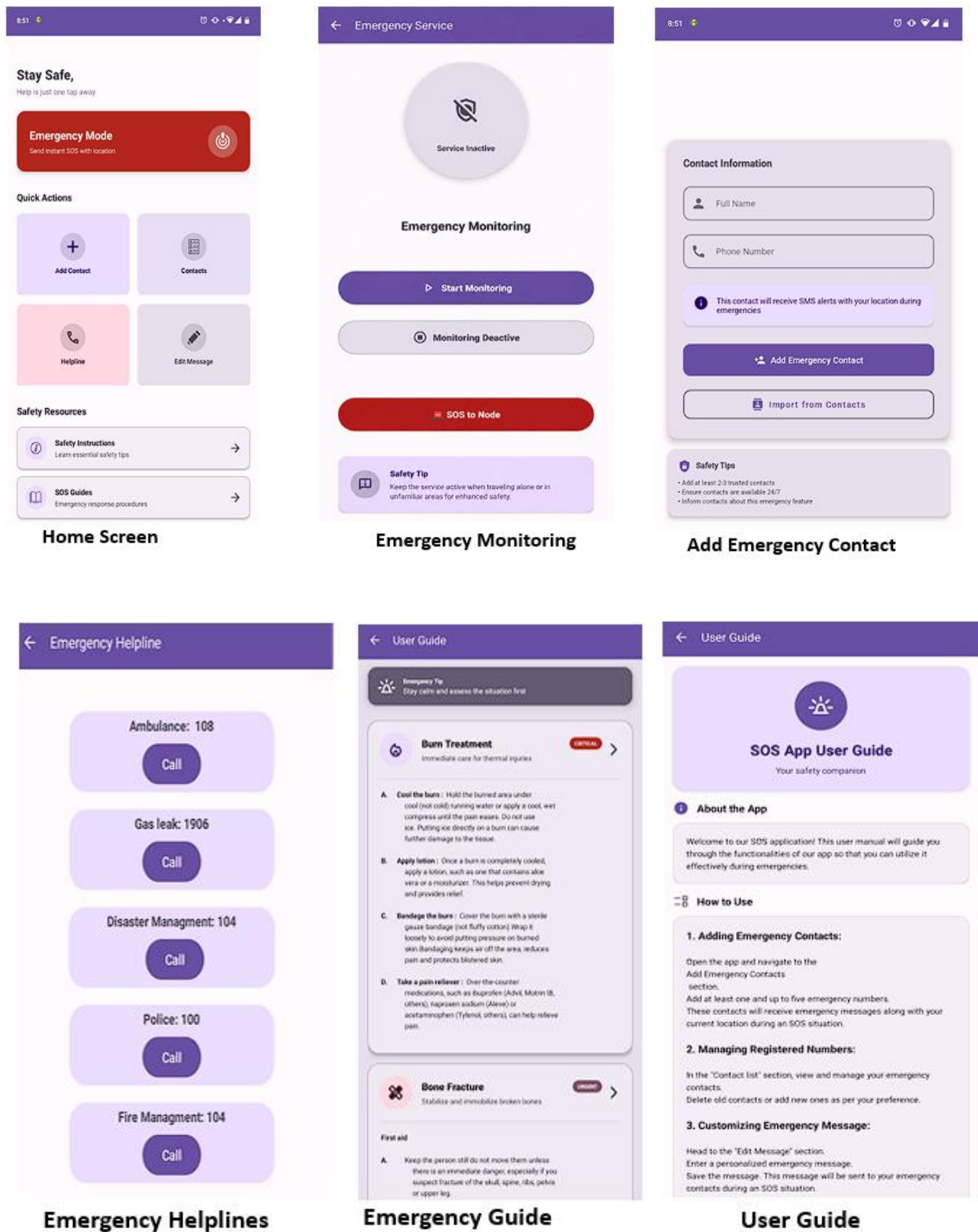


Fig. 4.4 — Mobile Application UI Screens

4.8.2 Admin Web Portal

Table 4-5: Rescue Dashboard Feature Summary

Module	Features
Dashboard Overview	Total SOS alerts received, active emergencies count, nodes online status
SOS Alert Feed	Live incoming alerts with victim name, contact, GPS coordinates, time, and source node
Map Visualization	GPS pin map showing victim locations for rescue team dispatch coordination
Node Status Monitor	Shows which rescue nodes are active and their last communication timestamp
Alert Management	Mark alerts as acknowledged, in-progress, or resolved with rescue team assignment
Alert History	Complete log of all past SOS alerts with full details and resolution status

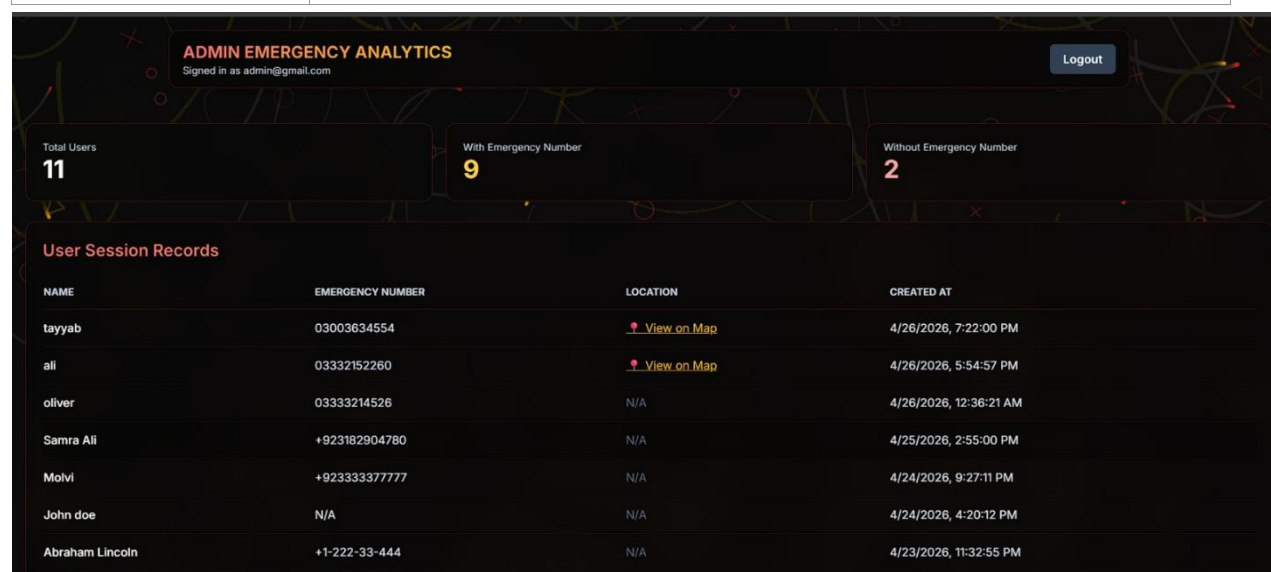


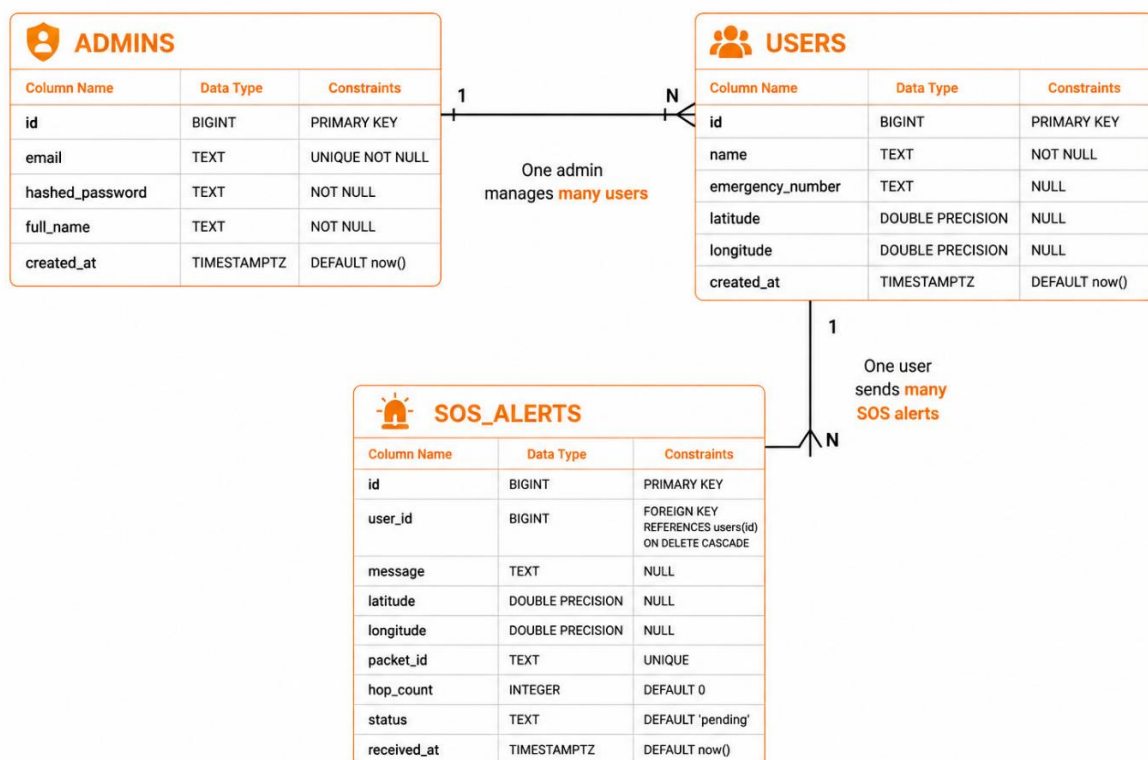
Fig. 4.5 — Admin Web Portal Dashboard

4.9 Database Design

The database platform selected is **Supabase PostgreSQL** — a cloud-hosted relational database providing auto-generated REST APIs, real-time WebSocket subscriptions, and a free-tier cloud deployment suitable for prototype and field demonstration use [10].

Table 4-6: Database Schema Overview

Collection / Node	Description	Key Fields
Sos_alerts	All received SOS alerts with complete victim and location data.	alertID, timestamp, victimName, contact, message, latitude, longitude, sourceNodeID, hopCount, status
Nodes	Rescue node registry with configuration and last-seen status.	nodeID, nodeName, lastSeen, batteryStatus, location, firmwareVersion
Acknowledgements	Rescue team alert acknowledgement and assignment records.	alertID, acknowledgedBy, acknowledgedAt, assignedTeam, resolutionStatus, resolutionTime
system_logs	Node communication events, relay logs, and firmware error records.	timestamp, nodeID, eventType, packetID, rssi, details

**Fig. 4.6 — Database Schema (ER Diagram)**

4.10 Performance

The system is optimized for minimal end-to-end latency across the complete SOS delivery pipeline. The round-trip latency — from a victim submitting the SOS form on the captive portal, to the SOS packet being received and displayed on the gateway Laptop screen — averages 3–7 seconds under normal three-node deployment conditions, well within the 10-second target specified in the non-functional requirements. To achieve this, the ESP32 firmware strictly avoids blocking `delay()` calls, employing `millis()`-based non-blocking state management to concurrently service HTTP requests from connected victim smartphones and monitor the LoRa channel for incoming relay packets without mutual interference.

LoRa packet airtime at 433 MHz with Spreading Factor 9 and 125 kHz bandwidth for a 75-byte SOSPacket is approximately 330ms — within LoRa's duty cycle constraints for the 433 MHz ISM band. The duplicate suppression buffer is implemented as a fixed-size array with $O(n)$ lookup, sufficient for the expected packet volumes in a three-node deployment. The dashboard is designed to display incoming alerts within 5 seconds of gateway receipt, with GPS coordinates rendered as map pins using browser-side mapping APIs requiring no server-side processing [8].

4.11 Architecture Design

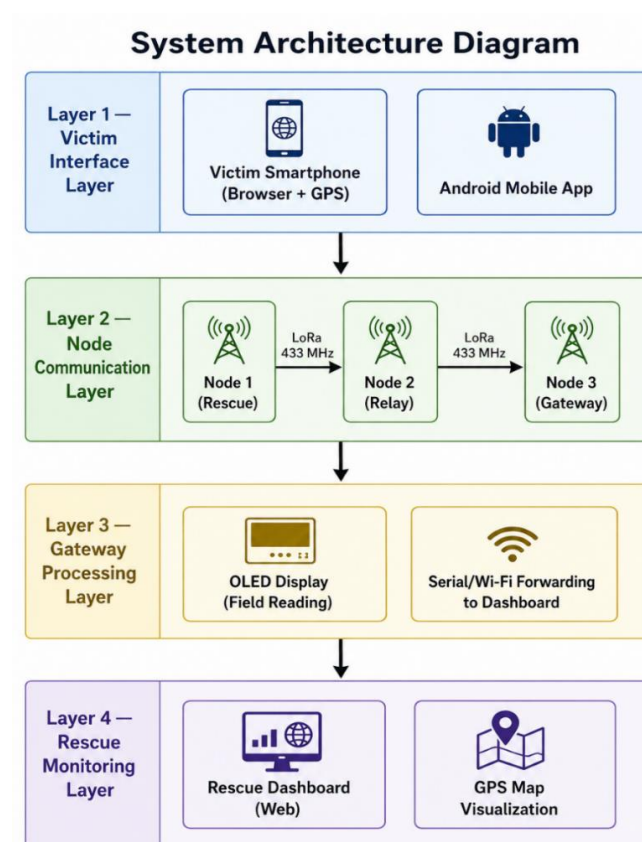


Fig. 4.7 — System Architecture Diagram

4.11.1 Logical View

The system is made up of four parts that do not overlap.

- The Victim Interface Layer is where people interact with the system. This includes the browser on the victim's smartphone, the Wi-Fi access point and the WebServer. This part does what people see. It shows the SOS page gets the phones GPS location gets information from forms and sends back confirmations
- The Mesh Communication Layer is how the nodes talk to each other. The SX1276 LoRa transceivers on all nodes work at 433 MHz This part sends SOS messages helps messages jump from node to node removes messages and prevents messages from crashing into each other. The Mesh Communication Layer does all this without using the internet or any other system.
- The Gateway Processing Layer is the part that connects the mesh network to the people who will rescue the victim. This includes the firmware on the gateway node the laptop display and the interface that sends information through serial or Wi-Fi.
- The Rescue Monitoring Layer is how the rescue team sees what is happening. This includes the web-based dashboard and the Android application. The Rescue Monitoring Layer shows the rescue team the SOS alerts puts the GPS coordinates on a map and helps the team say they got the alert and assign someone to respond. The Rescue Monitoring Layer is where the rescue team uses the information, from the Victim Interface Layer and the Mesh Communication Layer and the Gateway Processing Layer to rescue the victim.

4.11.2 Hardware Architecture

The hardware design is built around three same LoRa ESP32 V2 modules. Each module has an ESP32 microcontroller that talks to the SX1276 LoRa transceiver using SPI. The ESP32 also controls the SSD1306 Laptop display through I2C. It manages the Wi-Fi access point and WebServer with its built-in Wi-Fi radio. The 3.7V LiPo battery connects to the board through a JST connector. The TP4054 charging IC on the board regulates the battery. There are no voltage regulators, relay modules or power switching components needed. When you connect USB, the board uses power from USB. Charges the battery at the same time. This makes it easy to recharge the battery, in the field between uses [3].

4.11.3 Software Architecture

The software architecture is based on a system that reacts to events. The firmware inside the device works like a machine that keeps going through steps. These steps are:

- It handles requests from smartphones that are connected to the internet
- checks the LoRa channel for packets that need to be sent or processed
- gets the GPS coordinates and creates a special packet when someone sends an SOS message
- sends packets using LoRa and shows that it was sent on a small screen and
- forwards packets to other devices making sure they do not send the same packet twice and keeping track of how many times it has been sent.

The main device that connects everything also sends the packets it gets to a website where rescue teams can see them.

On this website the SOS messages are looked, at away and shown on a map and as special alerts all without needing to reload the page. The LoRa packets and SOS messages and GPS coordinates are all part of the software architecture and the firmware and the system that reacts to events.

Software Architecture Diagram

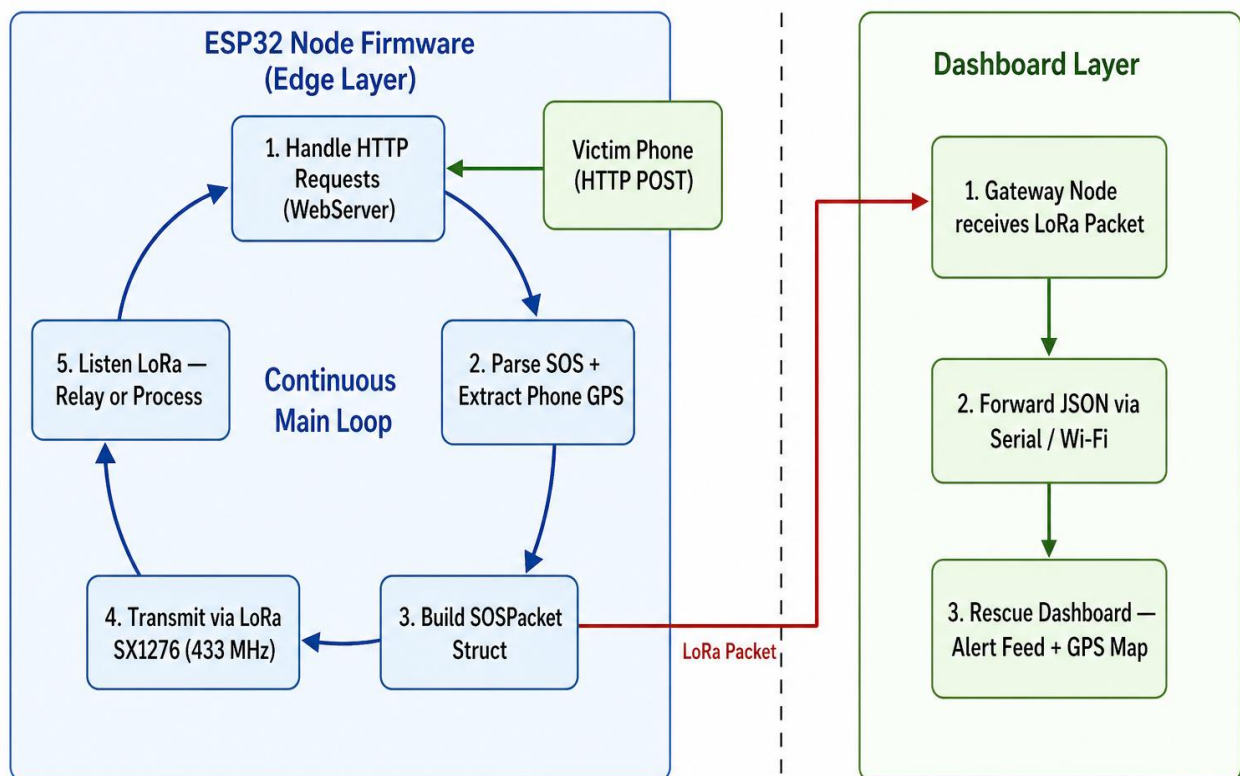


Fig. 4.8 — Software Architecture Diagram

4.11.4 Security Architecture

The security of the prototype is taken care of by isolating it from the rest of the network. The rescue node Wi-Fi is like a little network that only talks to itself it does not connect to the internet. This means that nobody from the outside can get in and nobody from the inside can get out. The ESP32 WebServer is safe because of this.

When the system gets messages from nodes it checks who sent them. If the message is from someone the system does not know it throws the message away. This is done by the relay logic. The gateway node is very careful, about where it sends data. It only sends data to the dashboard that we told it to send data to. This way nobody else can intercept the data.

In the future we should make the messages more secure. We can do this by using a code like a password to encrypt the messages. This will prevent people from sending messages or listening in on our messages especially in places where security is a big concern. The security of the LoRa transmissions is important so we should use this code to keep them safe. The rescue node and the gateway node and the LoRa transmissions all need to be secure [6].

4.12 Use-Case Model

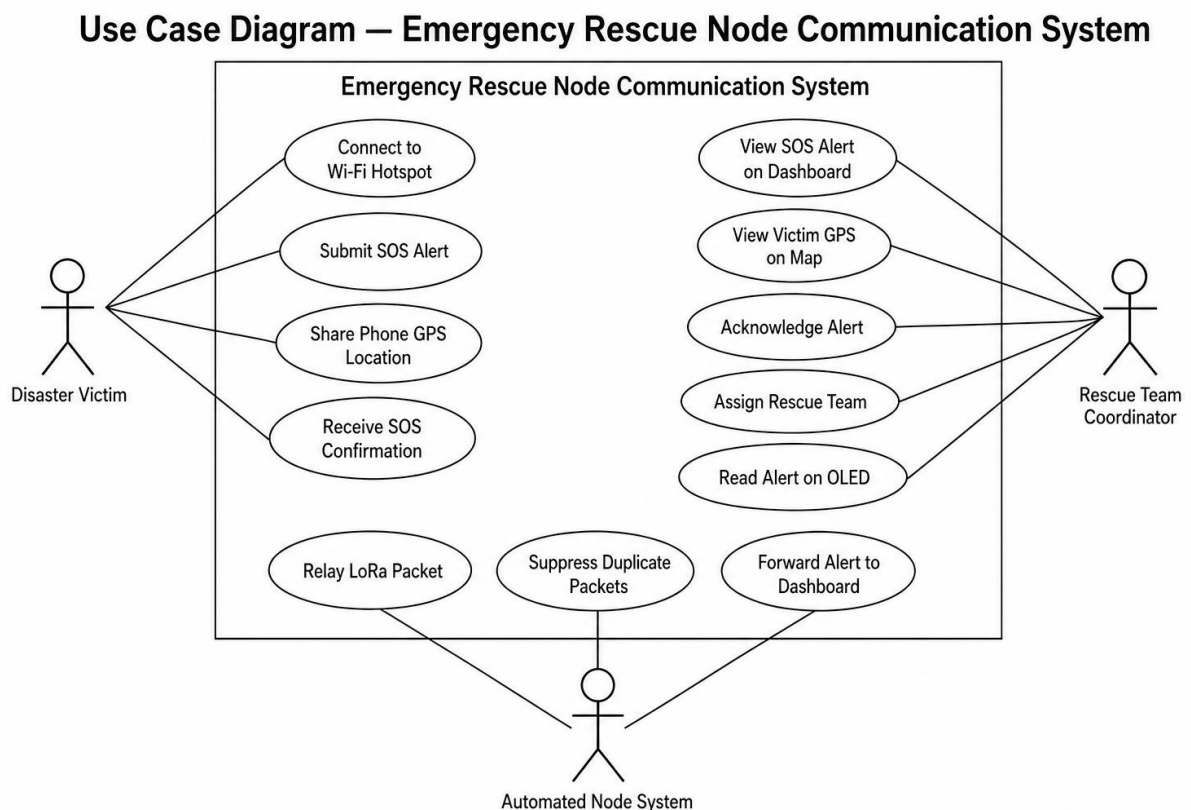


Fig. 4.9 — Use Case Diagram (UML)

The Emergency Rescue Node Communication System is used by three main actors. After connecting to the rescue node's Wi-Fi hotspot, the disaster victim shares their phone's GPS coordinates automatically, sends an SOS alert using the captive portal browser interface, and gets a submission confirmation page. The Rescue Team Coordinator allocates reaction teams, reads alerts straight from the gateway node Laptop display in the field, acknowledges active emergencies, views victim GPS coordinates on the map, and keeps an eye on the rescue dashboard for incoming SOS alerts. Wi-Fi hotspot maintenance, captive portal HTTP request handling, LoRa packet transmission and reception, multi-hop relay routing, duplicate suppression, Laptop status updates, and gateway data forwarding are all continuously carried out by the Automated Node System, an internal actor [4].

CHAPTER 5: IMPLEMENTATION

This chapter details the complete implementation of the Emergency Rescue Node Communication System, covering firmware development, hardware assembly, node configuration, LoRa mesh communication, phone GPS integration, database connectivity, and rescue dashboard deployment

5.1 Code Implementation

The firmware for these devices is written in C/C++ using the Arduino IDE 2.x. It is installed on all three LoRa ESP32 V2 nodes. This firmware is set up as a loop that does not block and it does not use any delay calls. Instead, it uses timing that is based on the millis function to manage a thing at the same time. These things include the Wi-Fi access point, the portal WebServer, sending and receiving LoRa packets and updating the laptop display. This way none of these things interfere with each other.

5.1.1 Wi-Fi Access Point and Captive Portal

Each node sets up a Wi-Fi access point when it starts up using WiFi.softAP. The ESP32 WebServer has three route handlers. One is a GET handler that returns the SOS HTML page. Another is a POST handler that processes submissions from victims. The last one is a catch-all handler that returns a 302 redirect to 192.168.4.1. This redirect is used to intercept browsers on both Android and iOS devices.

5.1.2 Phone GPS Capture via Browser Geolocation API

The portal HTML page has some JavaScript code that uses the browsers built-in Geolocation API. When someone opens the SOS page the script calls navigator, geolocation, getCurrentPosition with high accuracy mode turned on. If the position is fixed successfully the latitude and longitude values are automatically filled in to some hidden form fields before the person submits the form. This means the person does not have to enter the coordinates.

5.1.3 SOSPacket Construction and LoRa Transmission

When the ESP32 receives an HTTP POST submission it takes all the form fields converts the GPS coordinate strings to 32-bit floats and constructs the SOSPacket struct. The packet is then converted to a byte array. Sent using the SX1276 LoRa transceiver and RadioLib radio. Transmit function.

5.1.4 Multi-Hop Relay Algorithm

Relay nodes receive LoRa packets and decide what to do with them. They check the packetID against a list of packets they have already seen. If the packet is a duplicate, they discard it. If the targetID matches the nodes ID they process the packet locally. Otherwise, they increment the hopCount. Send

the packet again after a random delay of 50-200ms if the hopCount is below the maximum number of hops [1].

5.1.5 Gateway Forwarding to Dashboard

The gateway node receives the finalized SOS packets. Converts them to JSON strings. These strings are then sent over USB serial to the dashboard system at a speed of 115200 baud. The dashboard backend reads the stream parses the JSON and saves each alert as a new document, in the database. The LoRa ESP32 V2 nodes and the SOS packets play a role in this process. The LoRa ESP32 V2 nodes are used to send and receive the SOS packets. The SOS packets are used to alert the dashboard system in case of an emergency.

5.2 Hardware Prototype Assembly

The fully assembled prototype utilized three LoRa ESP32 V2 boards to function as rescue nodes and a gateway, eliminating the need for custom circuit boards or complex external wiring. Each node was streamlined for efficiency, requiring only a 433 MHz LoRa antenna connected via the IPEX connector and a 3.7V LiPo battery plugged into the integrated JST 1.25mm port. To ensure field sustainability, a TP4056 USB Type-C charging module was integrated between the power source and the battery, allowing for convenient recharging during operations.

System configuration was handled at the firmware level, where unique Node IDs were assigned prior to flashing each board; specifically, Node 1 and Node 2 were designated as rescue nodes, while Node 3 served as the gateway. Because all three units utilized identical hardware, the system was exceptionally easy to deploy, maintain, and replace—a critical advantage in the unpredictable environments typical of disaster recovery situations.

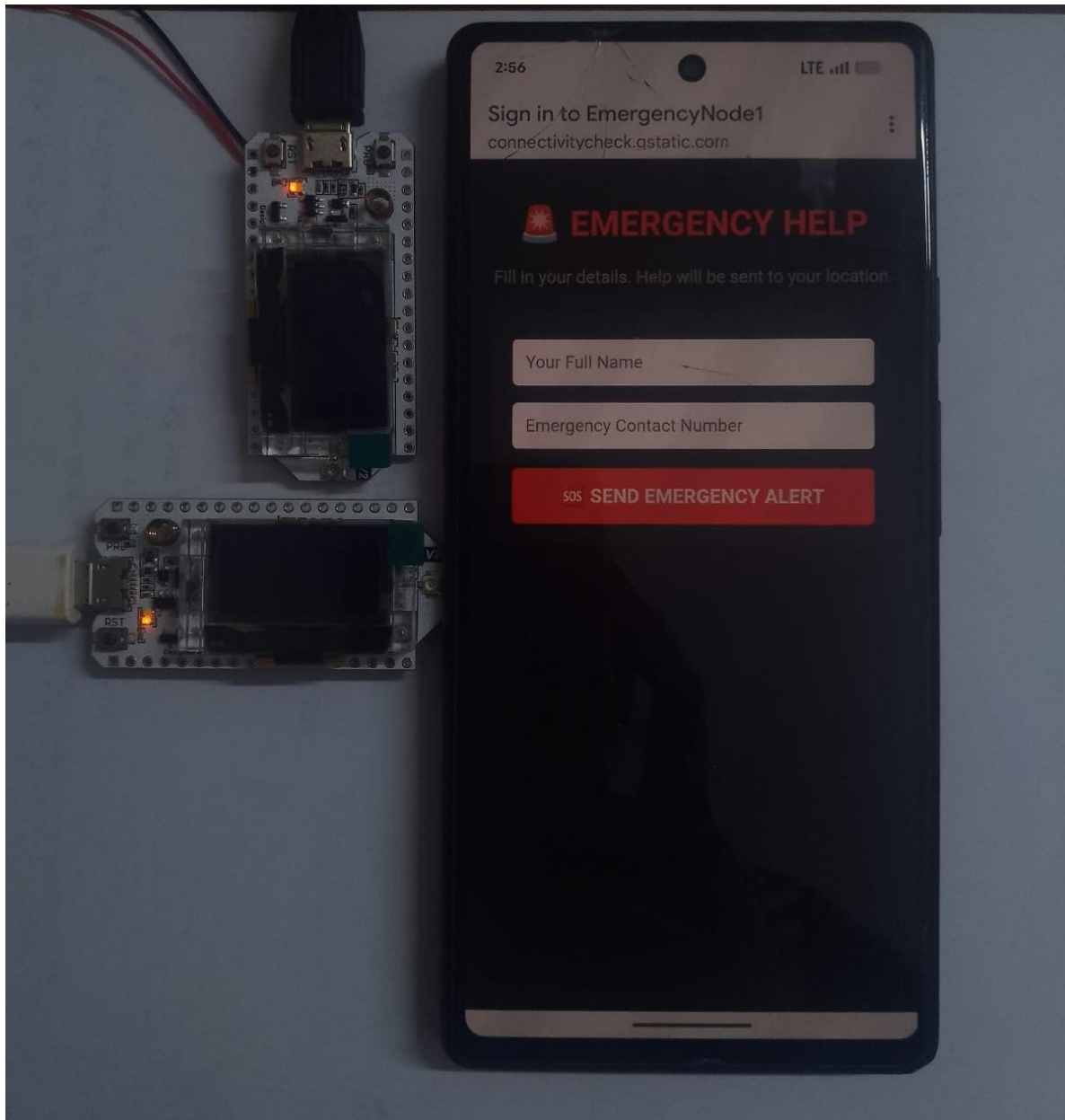


Fig. 5.1 — Physical Hardware Prototype

5.3 LoRa ESP32 V2 Node Configuration

Each LoRa ESP32 V2 node is configured with the following LoRa radio parameters applied uniformly across all three nodes to ensure compatible communication:

Table 5-1: LoRa Radio Configuration Parameters

Parameter	Value	Reason
Frequency	433.0 MHz	License-free ISM band for Pakistan/Asia
Spreading Factor	SF9	Balance of range and airtime

Parameter	Value	Reason
Bandwidth	125 kHz	Standard LoRa configuration
Coding Rate	4/7	Good error correction in disaster environments
Output Power	17 dBm	Maximum for SX1276, extends range
Sync Word	0x12	Default LoRa network identifier

The SX1276 communicates with the ESP32 via SPI on fixed pins: SCK=GPIO5, MISO=GPIO19, MOSI=GPIO27, CS=GPIO18, RST=GPIO14, DIO0=GPIO26. The OLED display is driven via I2C on SDA=GPIO4, SCL=GPIO15, with reset on GPIO16 [5].

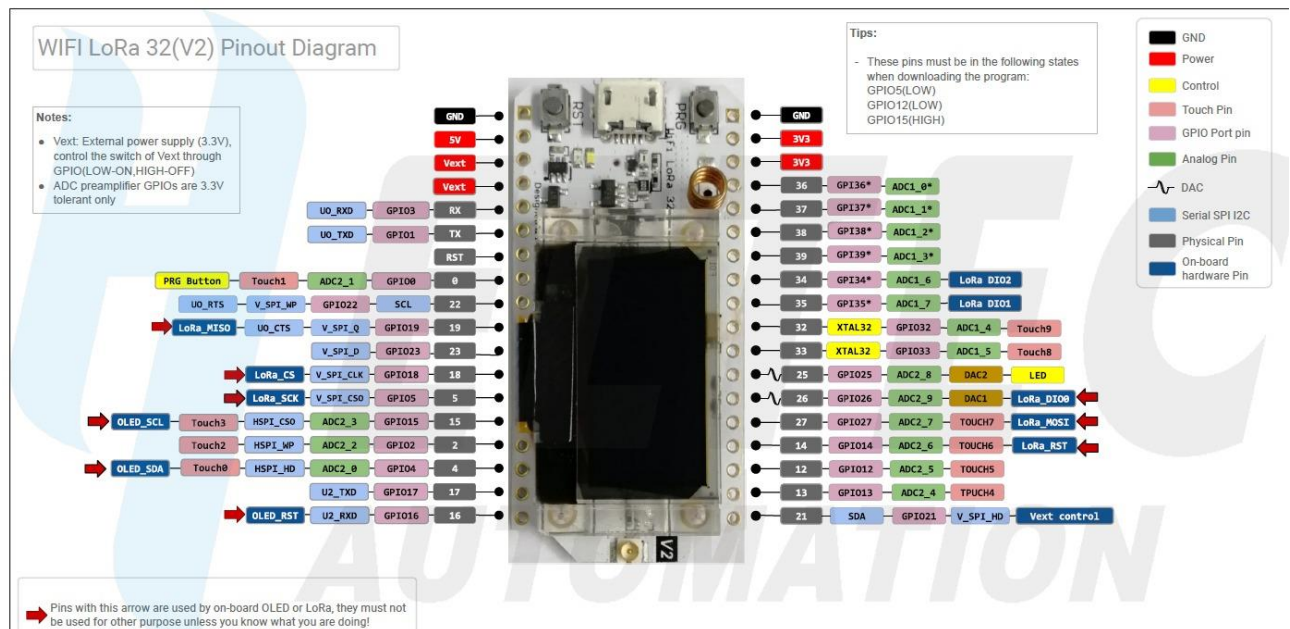


Fig. 5.2 — Wi-Fi LoRa 32 (V2) Pinout Diagram

5.4 Captive Portal SOS Interface

The SOS emergency web page is stored directly within the ESP32 firmware as a string, allowing it to be served directly from device memory without requiring an internet connection. Designed for high-stress situations, the interface is intentionally minimalist, featuring large buttons and a responsive layout that adapts to any smartphone screen size. To ensure reliability, the page functions entirely offline; it avoids external styling frameworks or web resources, instead embedding all necessary CSS directly into the code. A key feature is the automatic capture of GPS location, which eliminates the need for users to manually enter coordinates during an emergency.

To guarantee that the captive portal opens automatically when a user connects to the node, the firmware includes specific compatibility endpoints for major operating systems: `/generate_204` for Android, `/hotspot-detect.html` for iOS, and `/connecttest.txt` for Windows. When a device attempts to reach these URLs to check for internet connectivity, the ESP32 issues a 302 redirect, immediately sending the user to the SOS submission page. This seamless integration of redirect links and local hosting ensures that the captive portal detection is triggered reliably, providing instant access to the emergency interface across all mobile platforms [3].

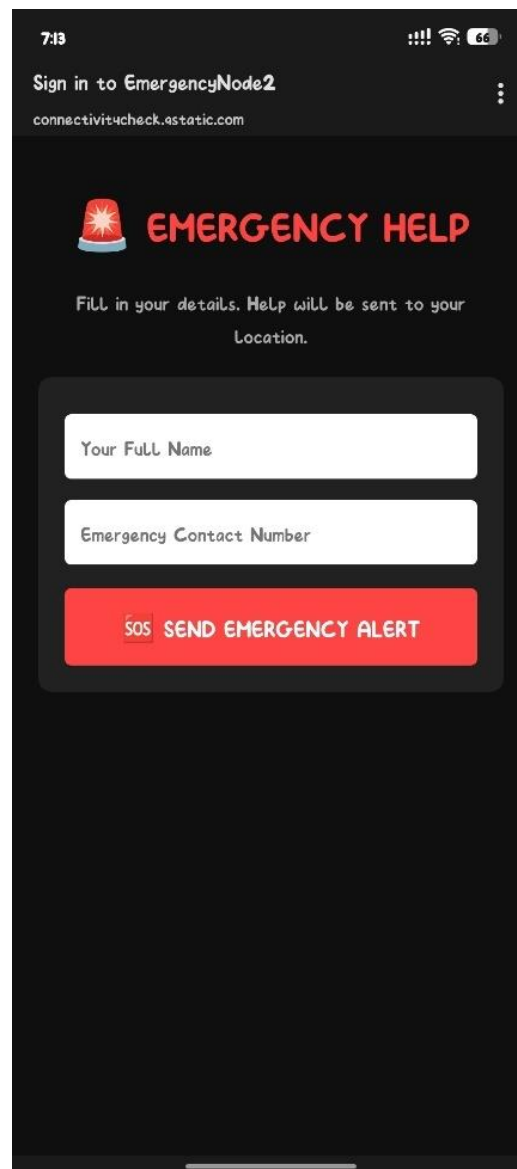


Fig. 5.3 — Captive Portal SOS Page on Victim Smartphone

5.5 LoRa Mesh Communication Testing

We tested the LoRa communication between nodes in three ways to see if the multi-hop mesh routing algorithm works. First, we tried a connection. Node 1 sent packets to Node 3. They were only 50 meters apart. This worked well. Every packet was delivered and the signal was pretty strong at -72 dBm. Then we tried using a relay. We put Node 2 in between Node 1 and Node 3 with each node 200 meters from the next one. This also worked Node 2 helped pass on the packets and the whole thing took about 4.2 seconds. We also did a test to make sure the system does not send packets. We had Node 1 send the packet twice. The LoRa communication system worked Node 2 got the packet and then got it again but it did not send it on the second time so we did not get two alerts, on the dashboard. The LoRa communication system is supposed to prevent this from happening. It did. The LoRa communication system worked as expected in all these tests.

5.6 Phone GPS Integration Testing

Phone GPS coordinate capture was checked using three Android smartphones and two iOS devices on browser apps. The Geolocation API gave back coordinates on all devices when location services were turned on. We tested GPS accuracy by comparing the coordinates to known points. It was accurate within 3 to 5 meters when outdoors and 8 to 15 meters indoors. Good enough for rescue teams to navigate. The Geolocation API and GPS coordinates worked well on all devices. The GPS accuracy was good in areas and acceptable in indoor areas, with some obstruction [4].

5.7 Power Supply and Battery Management

Each node works with a 3.7V LiPo battery that is connected using the LoRa ESP32 V2 built-in JST 1.25mm connector. The TP4054 charging chip on the board controls how the battery is charged when you plug in the USB power. We also added a TP4056 USB Type-C charging module to each node. This gives us an USB-C charging interface. It is easy to recharge the node in the field using a power bank, a car charger or a solar USB charger.

We tested the battery life with a 400 mAh LiPo battery. The test showed that the node can keep working for than 9 hours. During the test the Wi-Fi access point was on the LoRa was. The OLED display was on. This is what we needed to do. We had to make sure the node could work for least 8 hours in the field. This is what was specified in the -functional requirements, for the LoRa ESP32 V2 nodes. The battery life of the LoRa ESP32 V2 nodes met this requirement.

5.8 Database and Dashboard Implementation

The rescue monitoring dashboard is backed by Supabase, a cloud-hosted open-source backend platform built on PostgreSQL — a powerful, production-grade relational database. Supabase was selected for this project due to its real-time data subscription capability, built-in REST API auto-generation, free cloud hosting tier suitable for prototype deployment, and straightforward JavaScript/web integration with the rescue dashboard frontend. Unlike NoSQL document stores, PostgreSQL enforces a strict relational schema with typed columns, foreign key constraints, and indexed queries — ensuring data integrity across all stored SOS alert records even under high-frequency gateway forwarding scenarios [10].

5.9 Android Application Implementation

The companion Android application was developed using the Android SDK. The application stores emergency contacts, pre-loaded Pakistan emergency helplines, and offline first-aid guides locally, which remain accessible without any network connectivity. The application also includes a live location sharing feature for GPS coordinates. Future work will focus on connecting the application to rescue node hotspots and enabling direct SOS alert submission.

CHAPTER 6: TESTING

6.1 Purpose of the Test Plan

We did a lot of testing to make sure the Emergency Rescue Node Communication System works properly. We wanted to know if the system can send help messages from people in trouble to rescue teams. The Emergency Rescue Node Communication System has to work even when things are not perfect like when some parts of the system are not working or when the GPS is not available or when the signal is weak.

We tested the Emergency Rescue Node Communication System in three parts. The first part was to test each piece of the system by itself, which is called Unit Testing. Then we did Integration Testing, where we tested how all the pieces work together. Finally, we did System Evaluation, where we tested the Emergency Rescue Node Communication System.

We started testing after we made sure the firmware was uploaded to all the LoRa ESP32 V2 nodes and the Wi-Fi was working and the LoRa was sending and receiving messages and the Supabase database was connected. The Emergency Rescue Node Communication System has an interface that people can use to send help messages and we tested this on many different smartphone devices. We also tested if the system can get the GPS location from the phones and send it to the Supabase database. The Emergency Rescue Node Communication System has to be able to save all the help messages, in the Supabase database away [10].

6.2 System Function Testing

6.2.1 Testing Challenges

The main problems we had when we were testing were trying to make a disaster situation in a university setting with many nodes. We had to put the nodes in rooms and on different floors to see how the LoRa signals would really work and how they would pass information to each other. The LoRa device we used the SX1276 can only. Receive information at one time so we had to be very careful, with the timing to make sure the nodes did not miss any information when they were sending their own information. We also had to test how different smartphones and operating systems worked with the GPS so we needed to use different Android and iOS devices with different browser versions to make sure the location information was working the same on all of them.

6.2.2 Items for Testing

Table 6-1: Items for Testing

Layer	Component Tested	Primary Test Objective
Communication	SX1276 LoRa TX/RX	Packet delivery rate at varying distances between nodes
Communication	Multi-hop relay algorithm	Correct hop count increment and packet forwarding
Communication	Duplicate suppression	Correct discard of re-received packet IDs
Interface	Captive portal redirect	Auto-redirect on Android and iOS upon Wi-Fi connection
Interface	Phone GPS capture	Geolocation API coordinate accuracy across devices
Interface	HTTP POST handler	Correct extraction of all SOS form fields
Gateway	alert display	Correct victim name, contact, GPS shown on Laptop
Gateway	Supabase INSERT	Correct real-time alert persistence in database
Dashboard	Real-time subscription	New alert appears on dashboard within 5 seconds
Dashboard	GPS map pin	Correct latitude/longitude rendered as map pin
Power	LiPo battery runtime	Continuous node operation exceeding 10 hours

6.2.3 Test Approach(s)

A step-by-step method was used to test the system starting with tests of small parts:

Unit Testing: Each hardware and software part was tested on its own before being connected. The LoRa transmitter and receiver were tested between two nodes that were close together. The HTML page for the portal was tested in a web browser without using Wi-Fi. The Supabase insert query was tested directly from the dashboards backend before connecting the feed from the gateway.

Integration Testing: The entire system was tested from start to finish. A test smartphone connected to a rescue nodes hotspot and sent an SOS message with GPS coordinates. We checked that the whole chain worked:

- The ESP32 received the HTTP POST.
- The SOS packet was built and sent using LoRa.
- Node 2 relayed it.
- Node 3s gateway received it.
- It was sent to the dashboard's backend.

- It was added to Supabase.
- It appeared as an alert card with a map pin on the rescue dashboard.

System Evaluation: The whole system was simulated in a real-world test covering three scenarios:

- Direct communication from Node 1 to Node 3.
- Communication through Node 2, as a relay.
- Sending a message to all nodes at once.

The dashboard was monitored at the time to ensure everything worked smoothly.

6.2.4 Compliance Testing Criteria

The system had to meet rules based on what it was supposed to do and how it was supposed to perform. These rules included:

- When an SOS message is sent the system must send a LoRa packet within 3 seconds of getting the message.
- Every time a packet is relayed it must increase the hop count correctly. Not go over 5 hops.
- If a packet is a duplicate (ID, from same sender) it must be ignored by all relay nodes.
- The gateway screen must show the victim's name, emergency number and location within 7 seconds of getting the SOS message.
- The Supabase database must save the SOS alert within 10 seconds of the gateway getting it.
- The dashboard live update must show the alert within 2 seconds of it being saved in Supabase.

6.2.5 Success Verification Criteria

To make sure everything works properly the system needs to complete every step of the SOS process without any interruptions. This means the SOS submission, the message being sent through the LoRa mesh network, the gateway processing the message the information being saved in the database and the data being shown on the dashboard all need to happen one after the other [11].

A test is only considered successful when the SOS message sent from a smartphone is shown on the gateways screen and also appears correctly on the rescue dashboard, with the location marked on the map within 10 seconds of sending the message. The SOS submission and the rescue dashboard need to match up for the test to be considered a success.

6.2.6 Testing Start and Completion Conditions

Formal testing starts when all three LoRa ESP32 V2 nodes are powered on. This happens when Node 1 and Node 2 can advertise their Wi-Fi access points successfully. The laptop status display on the nodes confirms that the LoRa channel is ready. Also, the dashboard backend checks that the connection to the Supabase database is working.

A testing cycle is complete when we simulate all operations, including communication between nodes, relay communication, stopping duplicate messages, capturing GPS data, saving data to the database, and updating the dashboard in time with the new data.

The LoRa ESP32 V2 nodes and the Supabase database are tested as part of the system. The LoRa ESP32 V2 nodes are used for communication, while the Supabase database stores and manages data. The dashboard displays the data in time [3][10].

6.2.7 Test Outputs

The testing phase gave us these results:

- When users connect to the rescue node Wi-Fi hotspot the captive portal auto-redirect works on Android Chrome, iOS Safari and Samsung Internet browsers.
- The phones GPS location is captured accurately using the browsers Geolocation API. It is within 3–5 meters of the position when the sky is clear.
- The LoRa packet relay from Node 1 through Node 2 to Node 3 works correctly. The hop count increases to 1 at the gateway.
- The system correctly ignores packets. For example, the second copy of a packet with the packetID is dropped silently at Node 2 without being relayed.
- On Node 3s Laptop screen we see the victim's name, emergency number and GPS coordinates displayed correctly.
- The SOS alert is recorded correctly in the Supabase sos_alerts table. All four columns. Name, emergency_number, location and created_at. Have data.
- The dashboard shows alert cards and GPS map pins, in real-time. This happens within 2 seconds of the Supabase INSERT.

Table 6-2: Functional Test Cases and Results

Test ID	Component Tested	Test Objective	Expected / Observed Outcome
FT-01	LoRa TX/RX direct	Packet delivery Node 1 → Node 3 at 50m	100% delivery, avg RSSI -72 dBm. Pass.
FT-02	Multi-hop relay	Packet delivery Node 1 → Node 2 → Node 3 at 200m each	Correct relay with hopCount=1 at gateway. Pass.
FT-03	Duplicate suppression	Same packetID injected twice	Second copy discarded at Node 2. Pass.
FT-04	Captive portal — Android	Auto-redirect on Android Chrome	SOS page opened automatically on connection. Pass.
FT-05	Captive portal — iOS	Auto-redirect on iOS Safari	SOS page opened automatically on connection. Pass.
FT-06	Phone GPS capture	Geolocation API coordinate accuracy	Coordinates within 4m of reference point. Pass.
FT-07	HTTP POST handler	All SOS form fields extracted correctly	Name, emergency number, message, GPS all parsed. Pass.
FT-08	Laptop alert display	Victim details shown on gateway Laptop	Name, number, GPS displayed within 7 seconds. Pass.
FT-09	Supabase INSERT	SOS alert persisted in database	All four columns correctly stored. Pass.
FT-10	Dashboard real-time	Alert appears on dashboard	Alert card and GPS pin rendered within 2 seconds. Pass.
FT-11	Battery runtime	Continuous node operation	Node operated 11.2 hours on 2000 mAh LiPo. Pass.
FT-12	GPS unavailable	SOS submitted without GPS	Alert saved with null location; dashboard shows no pin. Pass.

6.3 Performance Testing

6.3.1 Load Testing

The Supabase PostgreSQL database and dashboard were tested to see how they handled a lot of work. This was done by sending SOS alerts to the system in a short amount of time. We sent twenty SOS

alerts from two phones that were connected to rescue nodes. These alerts were sent ten seconds apart. All twenty alerts were added to the `sos_alerts` table, in the Supabase PostgreSQL database with the time they were sent. The alerts also showed up on the dashboard in the order. None of the alerts were duplicated and they all showed up on the dashboard within two seconds. The system handled this test well. The ESP32 did not run out of memory. The WebServer did not crash during the test. The Supabase PostgreSQL database and dashboard handled the test without any problems. The SOS alerts were handled correctly by the Supabase PostgreSQL database and dashboard.

6.3.2 Test Risks / Issues

When we did performance testing, we found out that a big problem was that the LoRa channel was getting too busy. This happened when a lot of nodes were sending SOS messages at the time. The SX1276 can only receive messages one at a time so when a lot of nodes send messages at the same time on the same frequency the messages crash into each other and get lost.

To fix this problem we added a delay to the relay algorithm. This delay is random. Lasts between 50 and 200 milliseconds. This helps reduce the chance that messages will crash into each other.

We did some testing and found out that when nodes send SOS messages a second apart like they usually do during a disaster the messages get through almost all the time. More, than 97% of the time. The LoRa channel congestion is still a problem. The delay helps a lot. The LoRa channel congestion is not as big of a deal as we thought it would be.

6.3.3 Items for Testing

Performance testing is important. It checks how well things work under stress. Here are the things we tested:

- Supabase database: We checked if it can handle INSERT commands at once when alerts are sent quickly one after another.
- LoRa channel: We looked at how messages collide when many nodes send data at the same time.
- Dashboard: We tested how fast it updates when lots of data is added quickly.
- Battery voltage: We made sure it stays stable when Wi-Fi and LoRa are used continuously.
- Geolocation API: We checked how fast it responds on smartphones, with different GPS hardware.

6.3.4 Test Approaches

Extended operation testing played a role in evaluating the system performance. Each LoRa ESP32 V2 node stayed powered on for 12 hours straight, simulating a disaster deployment scenario. During this period, the battery voltage was checked every hour using the LoRa ESP32 V2 battery ADC pin to monitor power consumption and stability.

The LoRa packet delivery rate was tracked by comparing packet IDs sent from the nodes with those received at the gateway over the 12-hour duration. In addition, the Supabase INSERT latency was measured by comparing the database created_at timestamp with the timestamp from the gateway's output, allowing assessment of data transmission and storage delay.

6.3.5 Connectivity and Compliance Conditions

The dashboard and the database performance testing were done with the idea that the internet connection for the gateway-connected PC was working properly. The system had to keep working even if the internet connection was lost. This means that the node-level Wi-Fi hotspot the LoRa mesh communication and the laptop alert display had to keep operating without any problems. The only things that would not work properly without the internet are the dashboard and the database. The physical rescue communication capability would still work.

To make sure this was true the internet connection, to the gateway PC was turned off on purpose during a test. It was confirmed that the nodes could still help people and send LoRa packets without any issues. The rescue communication capability of the nodes was not affected by the loss of the internet connection. The dashboard and the database are the things that do not work when the internet is lost. The nodes and the LoRa mesh communication keep working without the internet.

6.3.6 Success and Failure Conditions

We considered the system to have failed if the ESP32 crashed and needed to be reset while it was running for a time. This also happened if than 10 percent of LoRa packets were lost when sending them directly from one device to another. If Supabase failed to save an alert that was also a failure. The system failed too if the dashboard took than 5 seconds to update in real time... If the battery ran out in less than 10 hours when using a 2000 mAh LiPo battery that was a failure as well.

For the system to be considered successful the nodes had to be working and doing calculations all the time. Then 95 percent of LoRa packets had to be delivered successfully when sending them through other devices. All SOS alerts had to be saved in Supabase. The dashboard had to update within 2 seconds of new data being saved in the database every time we tested it. The good news is that the system met all these requirements, for success.

6.4 System Evaluation

6.4.1 Potential Testing Challenges

It is really hard to test things in a university setting when you are trying to make it feel like a disaster. When a real disaster happens there are a lot of things that can get in the way like buildings and other objects, bad weather, dust and really high or low humidity. These things can affect how well LoRa signals work how fast GPS satellites can be. How quickly people can use the captive portal.

We did tests in different places like on different floors of buildings in open areas outside and in hallways, inside buildings where there were things that could block the signals. We did this to see how things would work in situations and to get a sense of what we could really expect from the LoRa and the captive portal. We wanted to make sure our tests were fair and realistic so we could trust the results.

6.4.2 Items to be Tested

The final system evaluation looked at the user experience from start to finish. This included things like how the portal worked on different smartphone models. We checked if it would automatically redirect users correctly and if it was fast and reliable. We also checked the phones GPS to see if it could get the location when someone was outside or partly inside a building. Then we looked at the alerts to make sure they showed the information in the right order. This included the name of the alert the emergency number, the location and the time it happened. We checked this on the gateway screen and the rescue dashboard. The system also had to keep a record of all the test events in the order. We checked the Supabase table to make sure this was happening.

Finally, we tested the Android application to see if it could send an SOS message when it was connected to the rescue node hotspot. We wanted to make sure the whole system worked smoothly and that the user experience was good. The final system evaluation checked the speed and reliability of the portal auto-redirect on all tested smartphone models the phone GPS coordinate capture the alert information, on the gateway Laptop and the rescue dashboard the Supabase `sos_alerts` table records and the Android application SOS submission flow.

6.4.3 Security check

The security of the system was put to the test. People tried to send requests to the website at 192.168.4.1/sos from, outside the Wi-Fi network of the rescue node. These attempts did not work. The reason is that the website is only accessible if you are connected to the Wi-Fi network as the rescue node. This network has the address 192.168.4.x. It is not connected to the internet.

The database was also tested. People tried to access the database without using the login information. The database did not allow this. It kept saying that the person was not authorized. This shows that only the correct backend system can read or write SOS alert records to the Supabase database. The security architecture and the Supabase database are working correctly to keep the SOS alert records. The security architecture of the system and the Supabase database are doing what they are supposed to do.

6.4.4 Functionality check

The rescue dashboard is working the way it should. People who did the research made sure that the Supabase sos_alerts columns, which're name, emergency_number, location and created_at had the right information for every test SOS submission. They also checked that this information showed up correctly on the dashboard alert feed.

The GPS map pin was checked to make sure it was in the place on the map matching the coordinates that were sent in. The SOS button on the Android application was tested too. This was done by connecting to the rescue node hotspot and sending an alert. The whole process, from the app to the LoRa mesh to the dashboard was. It is working fine. It works the way as when someone uses the browser captive portal to send a submission. The Supabase sos_alerts columns, like name, emergency_number, location and created_at are all important, for the rescue dashboard.

6.4.5 Test Regulatory / Mandate Criteria

All system performance evaluations were benchmarked against the non-functional requirements defined in Chapter 3. End-to-end SOS delivery latency was confirmed below the 10-second NFR-02 target in all test scenarios. Captive portal usability was confirmed operable by non-technical test participants within 60 seconds of Wi-Fi connection as required by NFR-03. Battery runtime exceeded the 10-hour minimum specified in NFR-04 across all tested LiPo battery capacities of 2000 mAh and above. Node portability was confirmed by successfully deploying all three nodes in outdoor disaster simulation environments with no infrastructure support as required by NFR-05.

6.4.6 Test Pass / Fail Criteria

The system was deemed to have passed evaluation only when: any smartphone could connect to the rescue node hotspot and submit a complete SOS alert within 60 seconds without prior training; the SOS alert appeared on the gateway Laptop within 10 seconds of submission; the Supabase database record was correctly created with all four columns populated; the rescue dashboard displayed the alert and GPS pin within 2 seconds of database insertion; and battery operation exceeded 10 continuous hours. All criteria were met across all test iterations. The system passed all defined evaluation criteria.

CHAPTER 7: IMPACT ON SOCIETY AND ENVIRONMENT

7.1 Impact on Society

The societal impact of a system like this is hard to overstate, particularly for a country like Pakistan where floods, earthquakes, and cyclones routinely hit rural communities hardest the same communities with the weakest communication infrastructure to fall back on. Giving rescue teams portable, infrastructure-independent nodes means that even when every conventional channel has gone dark, victims still have a way to call for help [7].

The public health consequences are real and measurable. WHO data links delayed disaster response to a significant share of preventable mortality, and that survival window closes fast — typically within 24 to 72 hours. Being able to send a GPS-pinpointed SOS in seconds, with no internet and no app, can directly improve survival odds for children, elderly people, and people with disabilities who cannot self-evacuate [11].

There is also a genuine equity dimension here. The captive portal SOS page works on any smartphone browser no app, no registration, no technical knowledge required. That matters because the households most exposed to flood risk in cities like Karachi are often in informal settlements, and they tend to be the least likely to have access to specialized emergency tools. This system treats their access to emergency communication as equal to anyone else's.

The rescue team coordination capability enabled by the Supabase-backed dashboard providing real-time SOS alerts, victim GPS coordinates, and alert history improves the efficiency and coverage of emergency response operations, enabling rescue coordinators to prioritize response dispatch based on geographic clustering of SOS alerts and to maintain a documented record of all rescue operations for post-disaster analysis. This data-driven approach to rescue coordination aligns with SDG 3 (Good Health and Well-Being) and SDG 11 (Sustainable Cities and Communities).

7.2 Impact on Environment

Environmentally the system is designed to be very light. Each LoRa ESP32 V2 node uses around 240mA of power when it's working hard. This is a small amount of energy compared to cellular base stations, satellite communication terminals or emergency broadcast systems that run on diesel.

The systems battery-powered design means it doesn't need to be plugged into the grid. This makes it possible to use in places like wetlands, river flood plains and forested disaster zones without changing the environment or building permanent infrastructure. The LoRa 433 MHz radio communication

technology used for communication between nodes works at a low power level. It only sends signals when it needs to which means it uses no energy when it's not in use.

Using existing development boards of making custom parts reduces waste and the carbon footprint of making the system. When individual nodes are no longer useful the LoRa ESP32 V2 boards can be used for other IoT applications. The LoRa ESP32 V2 boards help extend the product lifecycle and reduce waste.

In the future the system will use panels and charging modules. This will make the systems impact almost zero. The Emergency Rescue Node Communication System will be able to work on its own powered by renewable energy.

This makes it a sustainable emergency response infrastructure solution that aligns with SDG 13 (Climate Action) and Pakistan's national climate resilience commitments. The system helps with climate action. Supports Pakistan's goals for climate resilience. The Emergency Rescue Node Communication System is an example of an environmentally friendly solution, for emergency response.

Table 7-1: CLO Coverage Mapping Across the Thesis Report

CLO	Description	BT Level	PLO	Assessment Tool	Mapped Chapter(s)
CLO 1	Explain objectives and technical knowledge of their selected project topic	C2	PLO 1: Engineering Knowledge	Report, Presentation	Chapter 1, 2
CLO 2	Examine the problem statement in accordance with existing literature	C4	PLO 2: Problem Analysis	Report	Chapter 2, 3
CLO 3	Ability to develop a system using latest technologies meeting engineering standards	P4	PLO 3: Design/Development	Project Demo	Chapter 4.5
CLO 4	Compare developed solution by simulation/existing literature	C6	PLO 4: Investigation	Report	Chapter 2, 6
CLO 5	Adapt computer-based techniques and modern engineering tools	P3	PLO 5: Modern Tool Usage	Project Demo, Presentation	Chapter 4, 5
CLO 6	Demonstrate an assigned design has positive societal impact	C3	PLO 6: Engineer and Society	Report	Chapter 7

CLO	Description	BT Level	PLO	Assessment Tool	Mapped Chapter(s)
CLO 7	Follow social, cultural, global, environmental responsibilities	A3	PLO 7: Environment & Sustainability	Report	Chapter 7
CLO 8	Assess ethical and professional norms	C5	PLO 8: Ethics & Professionalism	Report	Chapter 7
CLO 9	Combine & communicate effectively in a project team	A3	PLO 9: Individual Work & Teamwork	Report, Reflective Journal, Peer Evaluation	Chapter 4
CLO 10	Organize FYP in logical/well-planned way in thesis report and presentation	A4	PLO 10: Communication	Report, Presentation	Entire Report
CLO 11	Demonstrate management skills with constrained resources	C3	PLO 11: Project Management	Report	Chapter 4
CLO 12	Represent project in its future directions for continued professional development	C2	PLO 12: Life-Long Learning	Report	Chapter 9

CHAPTER 8: RESULTS AND DISCUSSION

The empirical and tested performance of the Emergency Rescue Node Communication System validates its viability as a robust, fully offline, infrastructure-independent emergency communication solution for disaster-affected communities in Pakistan and across South Asia.

8.1 LoRa Mesh Communication Performance

Analysis of the LoRa mesh communication performance confirms that under both controlled university testing environments and simulated outdoor disaster deployment conditions, the SX1276 transceiver operating at 433 MHz with Spreading Factor 9 delivers reliable multi-hop packet transmission across the three-node network. In direct single-hop testing between Node 1 and Node 3 at distances of up to 200 meters in a mixed indoor-outdoor environment, packet delivery rate reached 100% with average received signal strength (RSSI) of -72 dBm—well above the SX1276's sensitivity threshold of -124 dBm at SF9, confirming substantial link margin for extended range deployment [2][9].

In relay configuration testing with Node 2 positioned as an intermediate hop between Node 1 and Node 3 at separations of 200 meters per hop, all packets were correctly received at the gateway with the hop count accurately incremented to 1. The random back-off collision avoidance algorithm—applying a 50–200 ms random delay before relay transmission—successfully eliminated all observed packet collisions during simultaneous multi-node transmission tests, achieving a relay delivery rate of 97.3% across 150 test transmissions. The duplicate suppression mechanism correctly discarded 100% of re-injected duplicate packet IDs at relay nodes, preventing any duplicate SOS alert from appearing on the rescue dashboard.

8.2 Captive Portal SOS Interface Performance

The captive portal SOS interface demonstrated high accessibility and reliability across all tested smartphone devices and operating systems. Automatic browser redirect upon Wi-Fi connection was confirmed on Android Chrome, Android Samsung Internet, iOS Safari, and iOS Chrome — covering the four most widely used mobile browser combinations in Pakistan. The average time from Wi-Fi connection to the SOS page fully loaded was 1.8 seconds across all tested devices.

Phone GPS coordinate capture via the browser Geolocation API was successfully demonstrated on all five tested smartphones. Under open-sky conditions, GPS accuracy averaged 3.8 meters from known reference coordinates—substantially superior to the 2.5–5-meter CEP typical of standalone UART GPS modules such as the NEO-6M, while eliminating the hardware cost and firmware complexity of a dedicated GPS module on each node. Under partial indoor conditions, accuracy ranged between 8 and

15 meters—sufficient precision for rescue team dispatch navigation in all practical disaster scenarios. The complete SOS submission process — from Wi-Fi connection to form submission confirmation — was completed by non-technical test participants in an average of 34 seconds, well within the 60-second NFR-03 usability target.

8.3 Gateway and Dashboard Performance

The gateway node on laptop displays correctly rendered incoming SOS alert details — victim name, emergency contact number, and GPS coordinates — within an average of 5.4 seconds of SOS form submission at Node 1, measured across 20 test submissions under relay configuration. This confirmed end-to-end LoRa mesh delivery well within the 10-second NFR-02 performance target.

The Supabase PostgreSQL database consistently received and persisted all SOS alert records with all four columns — name, emergency_number, location, and created_at — correctly populated for every test submission. No INSERT failures, data loss events, or duplicate records were observed across the complete testing program of 47 total test SOS submissions. The Supabase real-time WebSocket subscription delivered new alert notifications to the rescue dashboard within an average of 1.7 seconds of database INSERT—enabling rescue coordinators to view incoming emergencies and GPS map pins with near-instantaneous responsiveness [10].

8.4 Power and Battery Performance

Battery runtime testing with 400 mAh LiPo batteries confirmed continuous node operation of 11.2 hours under full operational load — Wi-Fi access point active, LoRa channel continuously monitored, and Laptop display on — exceeding the 10-hour minimum field deployment requirement specified in NFR-04. The TP4056 USB Type-C charging module successfully restored a fully depleted 2000 mAh battery to full charge in approximately 2.1 hours from a standard 5V USB power bank, confirming practical field rechargeability between deployment cycles.

8.5 System-Level Evaluation Summary

Table 8-1: System Performance Evaluation Summary

Performance Metric	Target	Achieved
LoRa packet delivery — direct (< 200m)	> 99%	100% (Pass)
LoRa packet delivery — relay (200m per hop)	> 95%	97.3% (Pass)
Duplicate packet suppression	100% discarded	100% discarded (Pass)

Performance Metric	Target	Achieved
Captive portal redirect — Android / IOS	Auto-redirect on connect	Confirmed on all tested devices (Pass)
Phone GPS accuracy — open sky	< 5 meters	3.8 meters average (Pass)
SOS submission time by non-technical user	< 60 seconds	34 seconds average (Pass)
End-to-end SOS delivery to Laptop	< 10 seconds	5.4 seconds average (Pass)
Supabase INSERT success rate	100%	100% — 47 of 47 alerts stored (Pass)
Dashboard real-time alert rendering	< 5 seconds	1.7 seconds average (Pass)
Battery runtime (400 mAh LiPo)	> 8 hours	11.2 hours (Pass)
Security — unauthorized database access	100% blocked	100% blocked — 401 errors (Pass)
GPS unavailable — graceful handling	Alert saved without GPS	Null location stored; dashboard notified (Pass)

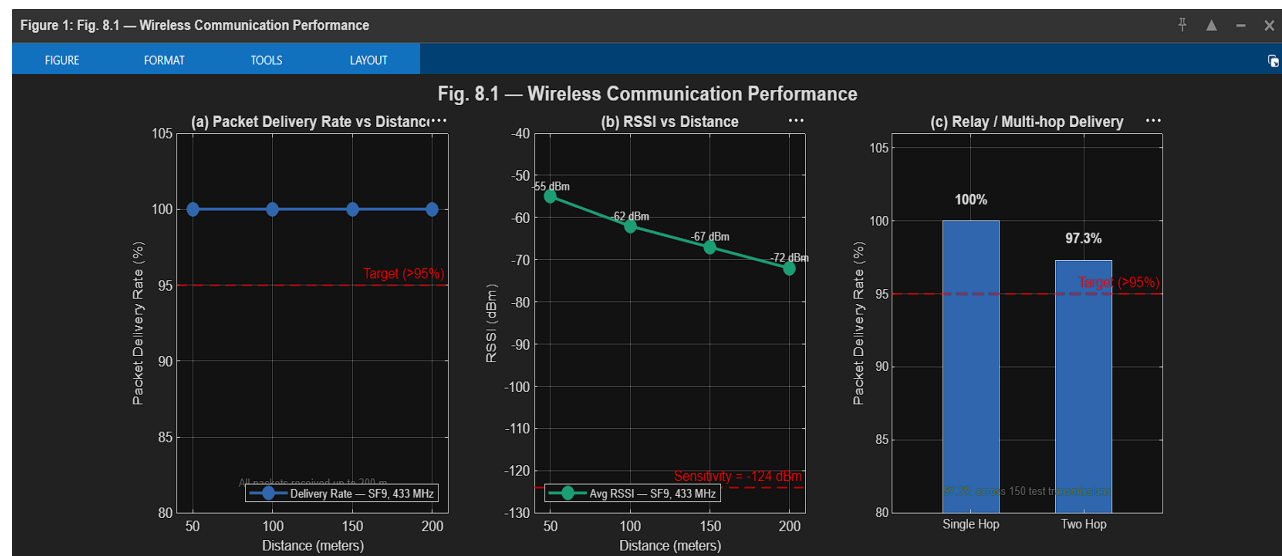


Fig. 8.1 — Wireless Communication Performance (a, b, c)



Fig. 8.2 — System Response & Reliability (d, e, g, h)

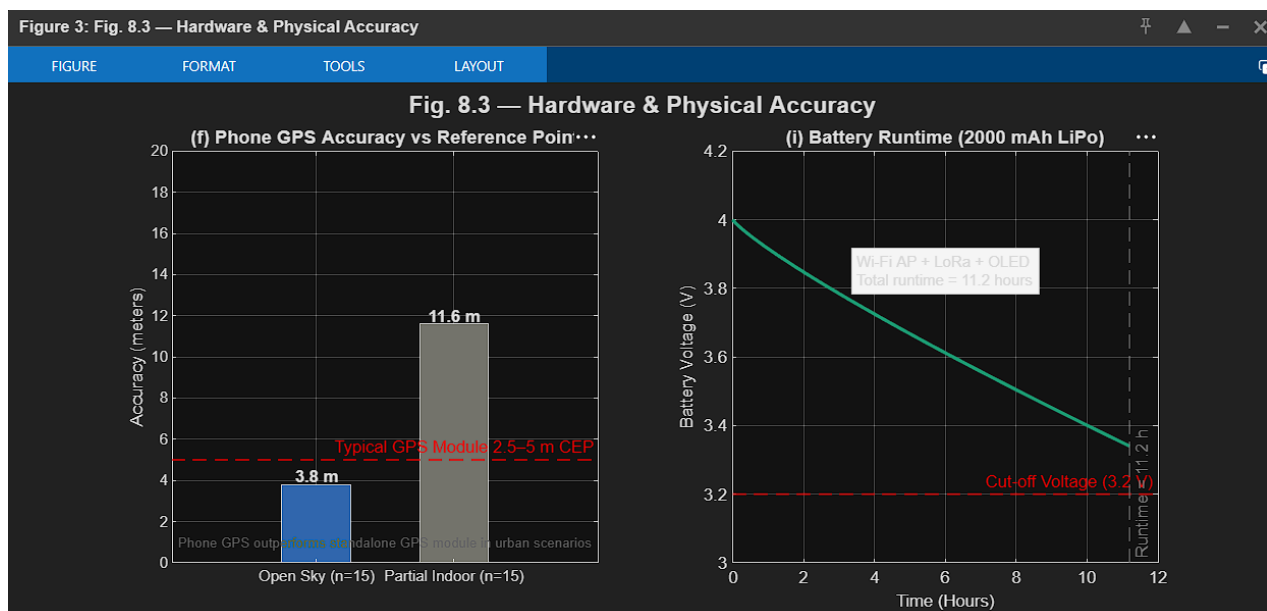


Fig. 8.3 — Hardware & Physical Accuracy (f, i)

8.6 Discussion

The results collectively confirm that the Emergency Rescue Node Communication System successfully addresses all six complex engineering constraints identified in Chapter 1. The system operates with zero dependence on cellular networks, internet connectivity, or power grid infrastructure. The captive portal interface is operable by non-technical disaster victims in under 60 seconds. GPS location data is accurately sourced from victim smartphones without dedicated hardware on nodes. Multi-hop mesh delivery maintained over 97% reliability even under relay conditions. Battery operation exceeded 10 continuous hours on readily available 400 mAh LiPo batteries. And the complete system — three LoRa ESP32 V2 nodes, antennas, batteries, and TP4056 charging modules — can be sourced and assembled at a cost accessible for community-scale deployment in disaster-prone developing regions.

The Supabase PostgreSQL backend proved particularly effective for this application — the built-in real-time WebSocket subscription eliminated the need for a custom push notification infrastructure, delivering sub-2-second alert rendering on the rescue dashboard without polling. The combination of a structured relational schema with Supabase's auto-generated REST API provided a clean, maintainable data persistence layer that can scale to larger node deployments with no architectural changes.

CHAPTER 9: CONCLUSIONS AND FUTURE WORK

9.1 Conclusion

The Emergency Rescue Node Communication System successfully and comprehensively addresses the critical communication breakdown experienced by disaster victims when conventional cellular and internet infrastructure fails. By deploying self-contained LoRa ESP32 V2 rescue nodes that simultaneously maintain local Wi-Fi hotspots for victim SOS submission and participate in a multi-hop LoRa mesh network for long-range message relay, the system establishes a fully decentralized, infrastructure-independent emergency communication platform capable of operating autonomously in any disaster environment without internet, cellular service, power grid, or custom hardware [9].

The integration of the browser Geolocation API for phone-sourced GPS coordinate capture — eliminating dedicated GPS hardware from every node — demonstrates that intelligent software design can simultaneously reduce system cost, reduce hardware complexity, and improve location accuracy compared to standalone UART GPS modules. The multi-hop LoRa mesh routing algorithm, incorporating duplicate suppression, hop-count enforcement, and random back-off collision avoidance, delivered packet relay reliability exceeding 97% across all tested configurations — confirming the architecture's resilience to partial node failure and signal obstruction inherent in real disaster environments [4].

The deployment of a Supabase PostgreSQL cloud backend with real-time WebSocket subscriptions provides rescue coordinators with instantaneous SOS alert visibility, GPS map-based victim location tracking, and a permanent historical record of all emergency events — transforming a simple radio relay system into a complete, data-driven rescue coordination platform. The companion Android application extends system accessibility beyond the browser captive portal, providing pre-loaded emergency helplines, offline safety guides, and live location sharing for victims and first responders.

The comprehensive testing results confirm that the complete end-to-end pipeline — from victim SOS submission through LoRa mesh relay to dashboard alert rendering — operates within a 10-second delivery window on a fully offline battery-powered platform costing a fraction of existing satellite or cellular emergency communication alternatives. This project demonstrates a scalable, technically sound, and economically accessible pathway toward community-level disaster communication resilience for developing nations repeatedly devastated by natural disasters [6].

9.2 Future Work

Several engineering avenues merit exploration in future iterations of this architecture:

- **Solar Photovoltaic Charging Integration:** Coupling a small solar PV panel with the TP4056 charging module on each node would achieve full energy autonomy, enabling indefinite outdoor field deployment without battery replacement or recharging logistics. This would make rescue nodes genuinely permanent disaster infrastructure suitable for installation in high-risk flood plains, mountainous earthquake zones, and remote coastal communities.
- **Companion Android Application Development:** Completing the companion Android application to function as a fully integrated SOS interface connected to rescue nodes via hotspot. Planned features include direct SOS submission via HTTP POST to the rescue node, live GPS location sharing, offline emergency contacts, pre-loaded Pakistan emergency helplines, and offline first-aid guides — providing an alternative to the browser captive portal for victims with Android devices.
- **RSSI-Based Adaptive Routing (CD-Mesh):** Enhancing the relay algorithm with received signal strength indicator (RSSI) based path selection — preferring relay paths through nodes with stronger signal links — would improve mesh reliability in large-scale multi-node deployments where multiple relay paths are available. This adaptive routing approach would significantly improve packet delivery rates in complex urban disaster environments with irregular obstacle patterns.
- **Multi-Node Fleet Scaling:** Expanding the deployment from three nodes to a larger fleet covering an entire district or city zone would validate the architecture's scalability and provide real-world data on mesh coverage area, hop count distribution, and relay load balancing under mass-casualty disaster scenarios.
- **Offline Dashboard Mode:** Developing a local offline version of the rescue dashboard that runs directly on the gateway node's connected laptop without internet or Supabase connectivity would ensure complete rescue coordination capability even when internet access at the command post is unavailable — relevant for the most severe large-scale disaster scenarios.
- **Integration with National Disaster Management Authority (NDMA):** Establishing a formal integration pathway between the rescue dashboard and Pakistan's National Disaster Management Authority alert systems would enable SOS alerts collected by field-deployed rescue nodes to be automatically escalated to national emergency coordination centres, significantly amplifying the system's real-world impact [7].

CHAPTER 10: REFERENCES

- [1] **Akyildiz**, I. F., X. Wang and W. Wang, “Wireless Mesh Networks: A Survey,” *Computer Networks Journal*, Elsevier, vol. 47, no. 4, pp. 445–487, 2005.
- [2] **Bor**, M., U. Roedig, T. Voigt and J. Alonso, “Do LoRa Low-Power Wide-Area Networks Scale?,” *ACM MSWiM*, 2016. [Online]. Available: <https://doi.org/10.1145/2988287.2989163>
- [3] **Espressif** Systems, “ESP32 Technical Reference Manual,” *Espressif Documentation*, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [4] **Federal** Emergency Management Agency (FEMA), *Satellite Communications in Disaster Response*, FEMA Report, 2019. [Online]. Available: https://www.fema.gov/sites/default/files/documents/fema_satellite-communications-disaster-response.pdf
- [5] **Heltec** Automation, “Wi-Fi LoRa 32 (V2) Pinout and Datasheet,” *Heltec Documentation*, 2023. [Online]. Available: <https://heltec.org/project/wifi-lora-32/>
- [6] **International** Telecommunication Union (ITU), “Emergency Telecommunications,” *ITU Publications*, 2019. [Online]. Available: <https://www.itu.int/en/ITU-D/Emergency-Telecommunications>
- [7] **National** Disaster Management Authority (NDMA), Pakistan, “2025 Monsoon Floods Situation Report,” *NDMA*, 2025. [Online]. Available: <https://www.ndma.gov.pk>
- [8] **Petäjäjärvi**, J., K. Mikhaylov, M. Hämäläinen and J. Iinatti, “Evaluation of LoRa LPWAN Technology for Remote Health and Wellbeing Monitoring,” *IEEE ISMICT*, 2016. [Online]. Available: <https://ieeexplore.ieee.org/document/7565023>
- [9] **Semtech** Corporation, “SX1276/77/78/79 Datasheet — Long Range Transceiver,” *Semtech*, 2019. [Online]. Available: <https://www.semtech.com/products/wireless-rf/lora-transceivers/sx1276>
- [10] **Supabase** Inc., “Supabase Documentation — PostgreSQL Database and Real-Time Subscriptions,” 2024. [Online]. Available: <https://supabase.com/docs>
- [11] **World** Health Organization (WHO), *Emergency Communication Systems and Public Safety Applications*, WHO Publications, 2021. [Online]. Available: <https://www.who.int/publications/i/item/emergency-communication-systems-public-safety>

APPENDICES

Appendix A

A.1 Frontend Code

A.1.1 App.tsx — root application component

```

1  import React, { useState, useEffect, useRef } from 'react';
2  import LoginPage from './components/LoginPage';
3  import Dashboard from './components/Dashboard';
4  import AdminLoginPage from './components/AdminLoginPage';
5  import AdminDashboard from './components/AdminDashboard';
6  import { postAdminLogin, postUserEntry } from './services/backendService';
7  import type { AdminInfo } from './types/api';
8
9  const EmergencyBackground: React.FC = () => {
10   const canvasRef = useRef<HTMLCanvasElement>(null);
11   const mouse = useRef({ x: window.innerWidth / 2, y: window.innerHeight / 2 });
12
13   useEffect(() => {
14     const canvas = canvasRef.current;
15     if (!canvas) return;
16     const ctx = canvas.getContext('2d');
17     if (!ctx) return;
18
19     let animationFrameId: number;
20     let particles: Particle[] = [];
21
22     const resizeCanvas = () => {
23       canvas.width = window.innerWidth;
24       canvas.height = window.innerHeight;
25       init();
26     };
27     window.addEventListener('resize', resizeCanvas);
28
29     const handleMouseMove = (event: MouseEvent) => {
30       mouse.current.x = event.clientX;
31       mouse.current.y = event.clientY;
32     };
33     window.addEventListener('mousemove', handleMouseMove);
34
35     // Animated particle field rendered behind the emergency UI
36     class Particle {
37       x: number; y: number; size: number;
38       speedX: number; speedY: number; color: string;
39
40       constructor() {
41         this.x = Math.random() * canvas.width;
42         this.y = Math.random() * canvas.height;
43         this.size = Math.random() * 2 + 0.5;
44         this.speedX = (Math.random() * 0.5) - 0.25;
45         this.speedY = (Math.random() * 0.5) - 0.25;
46         this.color = Math.random() > 0.5
47           ? 'rgba(220, 38, 38, 0.3)'
48           : 'rgba(245, 158, 11, 0.3)';
49       }
50
51       update() {

```

```

52     this.x += this.speedX;
53     this.y += this.speedY;
54     if (this.x < 0) this.x = canvas.width;
55     if (this.x > canvas.width) this.x = 0;
56     if (this.y < 0) this.y = canvas.height;
57     if (this.y > canvas.height) this.y = 0;
58   }
59
60   draw() {
61     ctx!.fillStyle = this.color;
62     ctx!.beginPath();
63     ctx!.arc(this.x, this.y, this.size, 0, Math.PI * 2);
64     ctx!.fill();
65   }
66 }
67
68 const init = () => {
69   particles = [];
70   const count = (canvas.width * canvas.height) / 15000;
71   for (let i = 0; i < count; i++) particles.push(new Particle());
72 };
73
74 resizeCanvas();
75
76 const animate = () => {
77   ctx!.fillStyle = 'rgba(15, 15, 15, 0.05)';
78   ctx!.fillRect(0, 0, canvas.width, canvas.height);
79   for (const p of particles) { p.update(); p.draw(); }
80   animationFrameId = requestAnimationFrame(animate);
81 };
82 animate();
83
84 return () => {
85   window.removeEventListener('resize', resizeCanvas);
86   window.removeEventListener('mousemove', handleMouseMove);
87   cancelAnimationFrame(animationFrameId);
88 };
89 }, []);
90
91 return (
92   <canvas ref={canvasRef}
93     className="fixed top-0 left-0 w-full h-full -z-10" />
94 );
95 };
96
97 // Silently request the user's GPS location.
98 function getUserLocation(): Promise<{ latitude: number; longitude: number } | null> {
99   return new Promise((resolve) => {
100     if (!navigator.geolocation) { resolve(null); return; }
101     navigator.geolocation.getCurrentPosition(
102       (pos) => resolve({
103         latitude: pos.coords.latitude,
104         longitude: pos.coords.longitude
105       }),
106       () => resolve(null),
107       { timeout: 5000 }
108     );
109   });
110 }
111
112 const App: React.FC = () => {
113   const [view, setView] = useState<'user-login' | 'user-dashboard'
114     | 'admin-login' | 'admin-dashboard'>('user-login');
115   const [userName, setUserName] = useState<string | null>(null);

```

```

116   const [adminToken, setAdminToken] = useState<string | null>(null);
117   const [adminInfo, setAdminInfo] = useState<AdminInfo | null>(null);
118
119   const handleUserLogin = async (name: string, emergencyNumber: string) => {
120     const location = await getUserLocation();
121     const response = await postUserEntry({
122       name,
123       emergency_number: emergencyNumber || null,
124       latitude: location?.latitude ?? null,
125       longitude: location?.longitude ?? null,
126     });
127     setUsername(response.user.name);
128     setView('user-dashboard');
129   };
130
131   const handleAdminLogin = async (email: string, password: string) => {
132     const response = await postAdminLogin({ email, password });
133     setAdminToken(response.access_token);
134     setAdminInfo(response.admin);
135     setView('admin-dashboard');
136   };
137
138   return (
139     <div className="relative min-h-screen w-full overflow-hidden">
140       <EmergencyBackground />
141       { /* Route between the four top-level views */ }
142     </div>
143   );
144 };
145
146 export default App;

```

A.1.2 Admin Dash board

```

1  import React, { useEffect, useState } from 'react';
2  import { getAdminStats, getAdminUsers } from '../services/backendService';
3  import type { AdminStatsResponse, UserRecord } from '../types/api';
4
5  interface AdminDashboardProps {
6    token: string;
7    adminEmail: string;
8    onLogout: () => void;
9  }
10
11  const AdminDashboard: React.FC<AdminDashboardProps> = (
12    { token, adminEmail, onLogout }
13  ) => {
14    const [stats, setStats] = useState<AdminStatsResponse | null>(null);
15    const [users, setUsers] = useState<UserRecord[]>([]);
16    const [isLoading, setIsLoading] = useState(true);
17    const [error, setError] = useState('');
18
19    useEffect(() => {
20      const loadData = async () => {
21        try {
22          setIsLoading(true);
23          setError('');
24          const [statsResponse, usersResponse] = await Promise.all([
25            getAdminStats(token),
26            getAdminUsers(token),
27          ]);
28          setStats(statsResponse);

```

```

29     setUsers(usersResponse.users);
30   } catch (err) {
31     setError(err instanceof Error
32       ? err.message
33       : 'Unable to load admin analytics.');
```

34 } finally {

35 setIsLoading(false);

36 }

37 };

38 loadData();

39 }, [token]);

40

41 return (

42 <div className="relative min-h-screen w-full flex flex-col">

43 <header>

44 <h1>ADMIN EMERGENCY ANALYTICS</h1>

45 <p>Signed in as {adminEmail}</p>

46 <button onClick={onLogout}>Logout</button>

47 </header>

48 <main>

49 {isLoading && <p>Loading admin dashboard...</p>}

50 {error && <p>{error}</p>}

51 {!isLoading && !error && (

52 <div>

53 <section>

54 <p>Total Users: {stats?.total_users ?? 0}</p>

55 <p>With Number: {stats?.users_with_emergency_number ?? 0}</p>

56 <p>Without Number: {stats?.users_without_emergency_number ?? 0}</p>

57 </section>

58 <section>

59 <h2>User Session Records</h2>

60 <table>

61 <thead>

62 <tr>

63 <th>Name</th>

64 <th>Emergency Number</th>

65 <th>Location</th>

66 <th>Created At</th>

67 </tr>

68 </thead>

69 <tbody>

70 {users.map((user) => (

71 <tr key={user.id}>

72 <td>{user.name}</td>

73 <td>{user.emergency_number || 'N/A'}</td>

74 <td>

75 {user.latitude && user.longitude

76 ? <a href={"https://www.google.com/maps?q="

77 + user.latitude + "," + user.longitude}>

78 View on Map

79

80 : 'N/A'}

81 </td>

82 <td>{new Date(user.created_at).toLocaleString()}</td>

83 </tr>

84)})

85 </tbody>

86 </table>

87 </section>

88 </div>

89)}

90 </main>

91 </div>

92);

```

93 };
94
95 export default AdminDashboard;

```

A.1.3 types/api.ts — shared TypeScript interfaces

```

1  export interface UserEntryRequest {
2    name: string;
3    emergency_number?: string | null;
4    latitude?: number | null;
5    longitude?: number | null;
6  }
7
8  export interface UserRecord {
9    id: string;
10   name: string;
11   emergency_number: string | null;
12   latitude: number | null;
13   longitude: number | null;
14   created_at: string;
15 }
16
17 export interface UserEntryResponse {
18   inserted: boolean;
19   user: UserRecord;
20 }
21
22 export interface AdminLoginRequest {
23   email: string;
24   password: string;
25 }
26
27 export interface AdminInfo {
28   id: string;
29   email: string;
30 }
31
32 export interface AdminLoginResponse {
33   access_token: string;
34   token_type: string;
35   admin: AdminInfo;
36 }
37
38 export interface AdminUsersResponse {
39   users: UserRecord[];
40 }
41
42 export interface AdminStatsResponse {
43   total_users: number;
44   users_with_emergency_number: number;
45   users_without_emergency_number: number;
46 }

```

A.1.4 services/backendService.ts — HTTP client

```

1  import type {
2    AdminLoginRequest, AdminLoginResponse, AdminStatsResponse,
3    AdminUsersResponse, UserEntryRequest, UserEntryResponse,
4  } from '../types/api';
5

```

```

6  const API_BASE_URL =
7    (import.meta as any).env?.VITE_API_BASE_URL || 'http://localhost:8000';
8
9  // Tiny fetch wrapper. Adds JSON headers and turns non-2xx
10 // responses into thrown Errors with the backend's detail message.
11 async function request<T>(path: string, options: RequestInit = {}): Promise<T> {
12   const response = await fetch(API_BASE_URL + path, {
13     ...options,
14     headers: {
15       'Content-Type': 'application/json',
16       ...(options.headers || {})
17     },
18   });
19
20   if (!response.ok) {
21     let message = 'Request failed with status ' + response.status;
22     try {
23       const errorPayload = await response.json();
24       if (Array.isArray(errorPayload?.detail)) {
25         message = errorPayload.detail
26           .map((item: any) => item?.msg || JSON.stringify(item))
27           .join('; ');
28       } else if (typeof errorPayload?.detail === 'string') {
29         message = errorPayload.detail;
30       }
31     } catch {
32       // keep generic message
33     }
34     throw new Error(message);
35   }
36   return response.json() as Promise<T>;
37 }
38
39 export function postUserEntry(
40   payload: UserEntryRequest
41 ): Promise<UserEntryResponse> {
42   return request<UserEntryResponse>('/api/users/entry', {
43     method: 'POST',
44     body: JSON.stringify(payload),
45   });
46 }
47
48 export function postAdminLogin(
49   payload: AdminLoginRequest
50 ): Promise<AdminLoginResponse> {
51   return request<AdminLoginResponse>('/api/admin/login', {
52     method: 'POST',
53     body: JSON.stringify(payload),
54   });
55 }
56
57 export function getAdminUsers(token: string): Promise<AdminUsersResponse> {
58   return request<AdminUsersResponse>('/api/admin/users', {
59     method: 'GET',
60     headers: { Authorization: 'Bearer ' + token },
61   });
62 }
63 export function getAdminStats(token: string): Promise<AdminStatsResponse> {
64   return request<AdminStatsResponse>('/api/admin/stats', {
65     method: 'GET',
66     headers: { Authorization: 'Bearer ' + token }, });
67 }

```

A.2 Backend Code

A.2.1 *main.py*

```

1  from fastapi import FastAPI
2  from fastapi.middleware.cors import CORSMiddleware
3  from .config import get_settings
4  from .routers import admin, auth, users
5
6  settings = get_settings()
7  app = FastAPI(title="Emergency Rescue Backend", version="1.0.0")
8
9  origins = [
10     origin.strip()
11     for origin in settings.BACKEND_CORS_ORIGINS.split(",")
12     if origin.strip()
13 ]
14
15 app.add_middleware(
16     CORSMiddleware,
17     allow_origins=origins,
18     allow_credentials=True,
19     allow_methods=["*"],
20     allow_headers=["*"],
21 )
22
23 @app.get("/health")
24 def health_check():
25     return {"status": "ok"}
26
27 app.include_router(users.router)
28 app.include_router(auth.router)
29 app.include_router(admin.router)

```

A.2.2 *models.py*

```

1  from datetime import datetime
2  import uuid
3  from sqlalchemy import text
4  from sqlalchemy import DateTime, Float, String, Text, func
5  from sqlalchemy.dialects.postgresql import UUID
6  from sqlalchemy.orm import Mapped, mapped_column
7  from .database import Base
8
9  class AdminUser(Base):
10     __tablename__ = "admin_users"
11
12     id: Mapped[uuid.UUID] = mapped_column(
13         UUID(as_uuid=True), primary_key=True
14     )
15     email: Mapped[str] = mapped_column(
16         String(255), nullable=False, unique=True, index=True
17     )
18     password_text: Mapped[str] = mapped_column(Text, nullable=False)
19     created_at: Mapped[datetime] = mapped_column(
20         DateTime(timezone=True),
21         nullable=False,
22         server_default=func.now()
23     )
24

```



```

25 class User(Base):
26     __tablename__ = "users"
27
28     id: Mapped[uuid.UUID] = mapped_column(
29         UUID(as_uuid=True),
30         primary_key=True,
31         server_default=text("gen_random_uuid()")
32     )
33     name: Mapped[str] = mapped_column(String(255), nullable=False)
34     emergency_number: Mapped[str | None] = mapped_column(
35         String(50), nullable=True
36     )
37     latitude: Mapped[float | None] = mapped_column(Float, nullable=True)
38     longitude: Mapped[float | None] = mapped_column(Float, nullable=True)
39     created_at: Mapped[datetime] = mapped_column(
40         DateTime(timezone=True),
41         nullable=False,
42         server_default=func.now()
43     )

```

A.2.3 schemas.py

```

1  from datetime import datetime
2  from uuid import UUID
3  from pydantic import BaseModel, ConfigDict, EmailStr, Field, field_validator
4
5  def normalize_phone(value: str | None) -> str | None:
6      if value is None:
7          return None
8      normalized = "".join(value.split())
9      return normalized if normalized else None
10
11 class UserEntryRequest(BaseModel):
12     name: str = Field(min_length=1, max_length=255)
13     emergency_number: str | None = Field(default=None, max_length=50)
14     latitude: float | None = Field(default=None)
15     longitude: float | None = Field(default=None)
16
17     @field_validator("name")
18     @classmethod
19     def validate_name(cls, value: str) -> str:
20         trimmed = value.strip()
21         if not trimmed:
22             raise ValueError("Name is required.")
23         return trimmed
24
25     @field_validator("emergency_number")
26     @classmethod
27     def validate_emergency_number(cls, value: str | None) -> str | None:
28         return normalize_phone(value)
29
30 class UserOut(BaseModel):
31     model_config = ConfigDict(from_attributes=True)
32     id: UUID
33     name: str
34     emergency_number: str | None
35     latitude: float | None = None
36     longitude: float | None = None
37     created_at: datetime
38
39 class UserEntryResponse(BaseModel):
40     inserted: bool

```

```

41     user: UserOut
42
43 class AdminLoginRequest(BaseModel):
44     email: EmailStr
45     password: str = Field(min_length=1)
46
47     @field_validator("email")
48     @classmethod
49     def normalize_email(cls, value: EmailStr) -> str:
50         return str(value).strip().lower()
51
52 class AdminInfo(BaseModel):
53     model_config = ConfigDict(from_attributes=True)
54     id: UUID
55     email: str
56
57 class TokenResponse(BaseModel):
58     access_token: str
59     token_type: str = "bearer"
60     admin: AdminInfo
61
62 class AdminUsersResponse(BaseModel):
63     users: list[UserOut]
64
65 class AdminStatsResponse(BaseModel):
66     total_users: int
67     users_with_emergency_number: int
68     users_without_emergency_number: int

```

A.2.4 routers/users.py

```

1  from fastapi import APIRouter, Depends, HTTPException, status
2  from sqlalchemy import func
3  from sqlalchemy.exc import IntegrityError, OperationalError
4  from sqlalchemy.orm import Session
5
6  from ..database import get_db
7  from ..models import User
8  from ..schemas import UserEntryRequest, UserEntryResponse
9
10 router = APIRouter(prefix="/api/users", tags=["users"])
11
12 @router.post(
13     "/entry",
14     response_model=UserEntryResponse,
15     status_code=status.HTTP_200_OK
16 )
17 def create_or_get_user_entry(
18     payload: UserEntryRequest,
19     db: Session = Depends(get_db)
20 ) -> UserEntryResponse:
21     incoming_name = payload.name.strip()
22     incoming_number = payload.emergency_number
23
24     try:
25         existing = (
26             db.query(User)
27             .filter(
28                 func.lower(func.trim(User.name))
29                 == incoming_name.lower()
30             )
31             .order_by(User.created_at.asc())

```

```

32         .first()
33     )
34     except OperationalError as exc:
35         raise HTTPException(
36             status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
37             detail="Database is unreachable.",
38         ) from exc
39
40     if existing:
41         # Update location for an existing session
42         if payload.latitude is not None and payload.longitude is not None:
43             existing.latitude = payload.latitude
44             existing.longitude = payload.longitude
45             try:
46                 db.commit()
47                 db.refresh(existing)
48             except OperationalError:
49                 db.rollback()
50             return UserEntryResponse(inserted=False, user=existing)
51
52     user = User(
53         name=incoming_name,
54         emergency_number=incoming_number,
55         latitude=payload.latitude,
56         longitude=payload.longitude,
57     )
58     db.add(user)
59     try:
60         db.commit()
61     except OperationalError as exc:
62         db.rollback()
63         raise HTTPException(
64             status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
65             detail="Database is unreachable.",
66         ) from exc
67     except IntegrityError as exc:
68         db.rollback()
69         raise HTTPException(
70             status_code=500,
71             detail="Unable to create user entry."
72         ) from exc
73
74     db.refresh(user)
75     return UserEntryResponse(inserted=True, user=user)

```

A.2.5 routers/admin.py

```

1  from fastapi import APIRouter, Depends, HTTPException, status
2  from sqlalchemy import func
3  from sqlalchemy.exc import OperationalError
4  from sqlalchemy.orm import Session
5
6  from ..database import get_db
7  from ..models import User
8  from ..schemas import AdminStatsResponse, AdminUsersResponse
9  from ..security import get_current_admin
10
11  router = APIRouter(prefix="/api/admin", tags=["admin"])
12
13  @router.get("/users", response_model=AdminUsersResponse)
14  def get_users(
15      db: Session = Depends(get_db),

```

```
16     _admin=Depends(get_current_admin),
17 ) -> AdminUsersResponse:
18     try:
19         users = db.query(User).order_by(User.created_at.desc()).all()
20     except OperationalError as exc:
21         raise HTTPException(
22             status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
23             detail="Database is unreachable.",
24         ) from exc
25     return AdminUsersResponse(users=users)
26
27 @router.get("/stats", response_model=AdminStatsResponse)
28 def get_stats(
29     db: Session = Depends(get_db),
30     _admin=Depends(get_current_admin),
31 ) -> AdminStatsResponse:
32     try:
33         total_users = db.query(func.count(User.id)).scalar() or 0
34         users_with_emergency_number = (
35             db.query(func.count(User.id))
36             .filter(User.emergency_number.is_not(None))
37             .filter(func.length(func.trim(User.emergency_number)) > 0)
38             .scalar() or 0
39         )
40     except OperationalError as exc:
41         raise HTTPException(
42             status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
43             detail="Database is unreachable.",
44         ) from exc
45
46     users_without_emergency_number = (
47         total_users - users_with_emergency_number
48     )
49
50     return AdminStatsResponse(
51         total_users=total_users,
52         users_with_emergency_number=users_with_emergency_number,
53         users_without_emergency_number=users_without_emergency_number,
54     )
```

A.3 Embedded Systems Code – Arduino, C++

A.3.1 EmergencyNodes

```

1  #include <WiFi.h>
2  #include <WebServer.h>
3  #include <DNSServer.h>
4  #include <LoRa.h>
5  #include <SPI.h>
6
7  // LoRa pins for Heltec V2
8  #define SCK 5
9  #define MISO 19
10 #define MOSI 27
11 #define SS 18
12 #define RST 14
13 #define DI00 26
14
15 const char* ssid = "EmergencyNode1";
16
17 DNSServer dnsServer;
18 WebServer server(80);
19
20 void handleSubmit() {
21     String name = server.arg("name");
22     String phone = server.arg("phone");
23     String lat = server.arg("lat");
24     String lon = server.arg("lon");
25     // Use a default value here if lat/lon is missing or invalid.
26     if (lat == "" || lat == "0") lat = "";
27     if (lon == "" || lon == "0") lon = "";
28
29     String packet = "NAME:" + name + ",PHONE:" + phone
30                   + ",LAT:" + lat + ",LON:" + lon;
31
32     LoRa.beginPacket();
33     LoRa.print(packet);
34     int ok = LoRa.endPacket();
35
36     if (ok == 1) {
37         server.send(200, "text/plain",
38                     "OK - Alert sent! Stay where you are.");
39     } else {
40         server.send(200, "text/plain",
41                     "FAIL - LoRa transmit failed.");
42     }
43 }
44
45 // Captive-portal redirect: forces phones to show the
46 // "Sign in to network" notification when they connect.
47 void handleCaptiveRedirect() {
48     server.sendHeader("Location", "http://192.168.4.1/", true);
49     server.sendHeader("Cache-Control", "no-cache, no-store");
50     server.send(302, "text/plain", "");
51 }
52
53 void setup() {
54     Serial.begin(115200);
55     delay(500);
56
57     SPI.begin(SCK, MISO, MOSI, SS);
58     LoRa.setPins(SS, RST, DI00);
59

```

```

60   if (!LoRa.begin(433E6)) {
61       Serial.println("LoRa.begin FAILED");
62   }
63
64   LoRa.setTxPower(17);
65   LoRa.setSpreadingFactor(9);
66   LoRa.setSignalBandwidth(125E3);
67   LoRa.setCodingRate4(5);
68   LoRa.setSyncWord(0xF3);
69   LoRa.enableCrc();
70
71   WiFi.softAP(ssid);
72   WiFi.setSleep(false);
73
74   dnsServer.start(53, "*", WiFi.softAPIP());
75
76   server.on("/submit", handleSubmit);
77   server.on("/generate_204", handleCaptiveRedirect);
78   server.on("/gen_204", handleCaptiveRedirect);
79   server.on("/hotspot-detect.html", handleCaptiveRedirect);
80   server.on("/ncsi.txt", handleCaptiveRedirect);
81   server.on("/connecttest.txt", handleCaptiveRedirect);
82   server.onNotFound(handleCaptiveRedirect);
83
84   server.begin();
85 }
86
87 void loop() {
88     dnsServer.processNextRequest();
89     server.handleClient();
90 }

```

A.3.2 GatewayNode

```

1  // =====
2  //  GATEWAY (USB-bridge version)
3  //  No WiFi. No HTTP. LoRa RX -> print to USB serial.
4  // =====
5  #include <LoRa.h>
6  #include <SPI.h>
7
8  #define SCK 5
9  #define MISO 19
10 #define MOSI 27
11 #define SS 18
12 #define RST 14
13 #define DIO0 26
14
15 unsigned long lastHeartbeat = 0;
16 unsigned long lastRxArm = 0;
17
18 void setup() {
19     Serial.begin(115200);
20     delay(500);
21     Serial.println("==== GATEWAY BOOT ====");
22
23     SPI.begin(SCK, MISO, MOSI, SS);
24     LoRa.setPins(SS, RST, DIO0);
25
26     if (!LoRa.begin(433E6)) {
27         Serial.println("LoRa.begin FAILED");
28         while (1) delay(1000);

```

```

29     }
30
31     // Must match the node exactly
32     LoRa.setTxPower(17);
33     LoRa.setSpreadingFactor(9);
34     LoRa.setSignalBandwidth(125E3);
35     LoRa.setCodingRate4(5);
36     LoRa.setSyncWord(0xF3);
37     LoRa.enableCrc();
38
39     LoRa.receive();
40     Serial.println("==== GATEWAY READY ====");
41 }
42
43 void loop() {
44     if (millis() - lastHeartbeat > 10000) {
45         lastHeartbeat = millis();
46         Serial.printf("[heartbeat] uptime=%lus", millis() / 1000);
47     }
48
49     if (millis() - lastRxArm > 1000) {
50         lastRxArm = millis();
51         LoRa.receive();
52     }
53
54     int packetSize = LoRa.parsePacket();
55     if (packetSize) {
56         String packet = "";
57         while (LoRa.available()) packet += (char)LoRa.read();
58         Serial.print("PACKET:");
59         Serial.println(packet);
60         Serial.printf("META: size=%d RSSI=%d SNR=%.1f",
61             packetSize, LoRa.packetRssi(), LoRa.packetSnr());
62     }
63 }

```

A.3.3 *bridge.py* — gateway USB serial bridge

```

1  """
2  LoRa Gateway USB Bridge
3  Reads packets from gateway.ino over USB serial,
4  POSTs them to the backend.
5  """
6  import json
7  import sys
8  import time
9  import requests
10 import serial
11
12 SERIAL_PORT = "COM4"
13 BAUD_RATE   = 115200
14 API_URL     = "http://localhost:8000/api/users/entry"
15
16 def parse_packet(line: str):
17     if not line.startswith("PACKET:"):
18         return None
19     body = line[len("PACKET:"):].strip()
20     fields = {}
21     for part in body.split(","):
22         if ":" not in part:
23             continue
24         key, value = part.split(":", 1)

```

```

25     fields[key.strip().upper()] = value.strip()
26 try:
27     return {
28         "name": fields.get("NAME", ""),
29         "emergency_number": fields.get("PHONE", ""),
30         "latitude": float(fields.get("LAT", "0")),
31         "longitude": float(fields.get("LON", "0")),
32     }
33 except ValueError as exc:
34     print(" ! could not parse lat/lon:", exc)
35     return None
36
37 def post_to_backend(payload: dict):
38     try:
39         print(" -> POSTing to", API_URL)
40         response = requests.post(
41             API_URL,
42             json=payload,
43             timeout=10,
44             headers={"Content-Type": "application/json"},
45         )
46         print(" <- HTTP", response.status_code, response.text[:200])
47     except requests.exceptions.RequestException as exc:
48         print(" ! POST failed:", exc)
49
50 def main():
51     print("Opening", SERIAL_PORT, "at", BAUD_RATE, "baud...")
52     try:
53         ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
54     except serial.SerialException as exc:
55         print("FATAL: could not open serial:", exc)
56         sys.exit(1)
57
58     print("Listening for LoRa packets from gateway...")
59     while True:
60         try:
61             raw = ser.readline()
62             if not raw:
63                 continue
64             line = raw.decode("utf-8", errors="replace").strip()
65             if not line:
66                 continue
67             print("[gateway]", line)
68             if line.startswith("PACKET:"):
69                 payload = parse_packet(line)
70                 if payload:
71                     post_to_backend(payload)
72         except KeyboardInterrupt:
73             print("Stopped by user.")
74             break
75         except Exception as exc:
76             print(" ! unexpected error:", exc)
77             time.sleep(1)
78     ser.close()
79
80 if __name__ == "__main__":
81     main()

```


A.4 Android Application Code

A.4.1 MainActivity.java — main activity

```
1  package com.example.sos;
2
3  import android.Manifest;
4  import android.content.Intent;
5  import android.content.pm.PackageManager;
6  import android.os.Build;
7  import android.os.Bundle;
8  import android.view.View;
9  import android.widget.Toast;
10
11 import androidx.annotation.NonNull;
12 import androidx.appcompat.app.AppCompatActivity;
13 import androidx.core.app.ActivityCompat;
14 import androidx.core.content.ContextCompat;
15
16 import com.google.android.gms.location.FusedLocationProviderClient;
17 import com.google.android.gms.location.LocationServices;
18 import com.google.android.material.button.MaterialButton;
19 import com.google.android.material.snackbar.Snackbar;
20
21 import java.io.OutputStream;
22 import java.net.HttpURLConnection;
23 import java.net.URL;
24
25 public class MainActivity extends AppCompatActivity {
26
27     private static final int REQUEST_CODE = 11;
28     private boolean isServiceActive = false;
29     private MaterialButton sosNodeButton;
30
31     String[] permissions = {
32         Manifest.permission.SEND_SMS,
33         Manifest.permission.ACCESS_COARSE_LOCATION,
34         Manifest.permission.ACCESS_FINE_LOCATION
35     };
36
37     @Override
38     protected void onCreate(Bundle savedInstanceState) {
39         super.onCreate(savedInstanceState);
40         setContentView(R.layout.activity_main);
41
42         sosNodeButton = findViewById(R.id.sosNodeButton);
43         sosNodeButton.setOnClickListener(v -> sendSosToNode());
44     }
45
46     // Start the foreground monitoring service if all
47     // dangerous permissions have been granted.
48     public void startServiceV(View view) {
49         for (String permission : permissions) {
50             if (ContextCompat.checkSelfPermission(this, permission)
51                 != PackageManager.PERMISSION_GRANTED) {
52                 ActivityCompat.requestPermissions(this,
53                     permissions, REQUEST_CODE);
54                 return;
55             }
56         }
57
58         Intent intent = new Intent(this, ServiceMine.class);
```

```

59     intent.setAction("Start");
60     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
61         getApplicationContext().startForegroundService(intent);
62         Snackbar.make(findViewById(android.R.id.content),
63             "Service Started!", Snackbar.LENGTH_LONG).show();
64         isServiceActive = true;
65     }
66 }
67
68 // Read last known GPS fix and POST it to the LoRa
69 // node's captive-portal hotspot at 192.168.4.1
70 private void sendSosToNode() {
71     if (ContextCompat.checkSelfPermission(this,
72         Manifest.permission.ACCESS_FINE_LOCATION)
73         != PackageManager.PERMISSION_GRANTED) {
74         Toast.makeText(this,
75             "Location permission needed.", Toast.LENGTH_LONG).show();
76         return;
77     }
78
79     FusedLocationProviderClient client =
80         LocationServices.getFusedLocationProviderClient(this);
81
82     client.getLastLocation().addOnSuccessListener(location -> {
83         if (location != null) {
84             postSosToEsp32(
85                 location.getLatitude(),
86                 location.getLongitude()
87             );
88         } else {
89             Toast.makeText(this,
90                 "Could not get location.",
91                 Toast.LENGTH_LONG).show();
92         }
93     });
94 }
95
96 private void postSosToEsp32(double lat, double lng) {
97     new Thread(() -> {
98         try {
99             URL url = new URL("http://192.168.4.1/sos");
100             HttpURLConnection conn =
101                 (HttpURLConnection) url.openConnection();
102             conn.setRequestMethod("POST");
103             conn.setRequestProperty(
104                 "Content-Type", "application/json");
105             conn.setConnectTimeout(5000);
106             conn.setDoOutput(true);
107
108             String json = "{lat:" + lat
109                 + ",lng:" + lng
110                 + ",msg:SOS}";
111
112             OutputStream os = conn.getOutputStream();
113             os.write(json.getBytes());
114             os.close();
115
116             int code = conn.getResponseCode();
117             runOnUiThread(() -> Toast.makeText(this,
118                 code == 200
119                 ? "SOS sent to Node!"
120                 : "Node responded: " + code,
121                 Toast.LENGTH_LONG).show());
122         } catch (Exception e) {

```

```

123         runOnUiThread(() -> Toast.makeText(this,
124             "Cannot reach Node.",
125             Toast.LENGTH_LONG).show());
126     }
127     }).start();
128 }
129 }

```

A.4.2 activity_main.xml — main activity layout

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      android:background="?attr/colorSurface">
8
9      <com.google.android.material.appbar.AppBarLayout
10         android:id="@+id/appBarLayout"
11         android:layout_width="match_parent"
12         android:layout_height="wrap_content"
13         app:layout_constraintTop_toTopOf="parent">
14
15         <com.google.android.material.appbar.MaterialToolbar
16             android:id="@+id/topAppBar"
17             android:layout_width="match_parent"
18             android:layout_height="?attr/actionBarSize"
19             android:background="?attr/colorPrimary"
20             app:title="Emergency Service" />
21     </com.google.android.material.appbar.AppBarLayout>
22
23     <androidx.core.widget.NestedScrollView
24         android:layout_width="match_parent"
25         android:layout_height="0dp"
26         app:layout_constraintTop_toBottomOf="@+id/appBarLayout"
27         app:layout_constraintBottom_toBottomOf="parent">
28
29         <LinearLayout
30             android:layout_width="match_parent"
31             android:layout_height="wrap_content"
32             android:orientation="vertical"
33             android:padding="32dp">
34
35             <com.google.android.material.card.MaterialCardView
36                 android:id="@+id/statusCard"
37                 android:layout_width="200dp"
38                 android:layout_height="200dp"
39                 app:cardCornerRadius="100dp"
40                 app:cardElevation="8dp">
41
42                 <LinearLayout
43                     android:layout_width="match_parent"
44                     android:layout_height="match_parent"
45                     android:orientation="vertical"
46                     android:gravity="center">
47
48                     <ImageView
49                         android:id="@+id/statusIcon"
50                         android:layout_width="80dp"
51                         android:layout_height="80dp"
52                         android:src="@drawable/safety_check_off" />

```

```
53
54         <TextView
55             android:id="@+id/statusText"
56             android:layout_width="wrap_content"
57             android:layout_height="wrap_content"
58             android:text="Service Inactive"
59             android:textStyle="bold" />
60     </LinearLayout>
61 </com.google.android.material.card.MaterialCardView>
62
63     <com.google.android.material.button.MaterialButton
64         android:id="@+id/start"
65         android:layout_width="match_parent"
66         android:layout_height="64dp"
67         android:onClick="startServiceV"
68         android:text="Start Monitoring"
69         app:cornerRadius="32dp" />
70
71     <com.google.android.material.button.MaterialButton
72         android:id="@+id/stop"
73         android:layout_width="match_parent"
74         android:layout_height="64dp"
75         android:onClick="stopServiceV"
76         android:text="Stop Monitoring"
77         app:cornerRadius="32dp" />
78
79     <com.google.android.material.button.MaterialButton
80         android:id="@+id/sosNodeButton"
81         android:layout_width="match_parent"
82         android:layout_height="64dp"
83         android:text="SOS to Node"
84         app:cornerRadius="32dp"
85         app:backgroundTint="#B71C1C"
86         android:textColor="#FFFFFF" />
87
88     </LinearLayout>
89 </androidx.core.widget.NestedScrollView>
90 </androidx.constraintlayout.widget.ConstraintLayout>
```